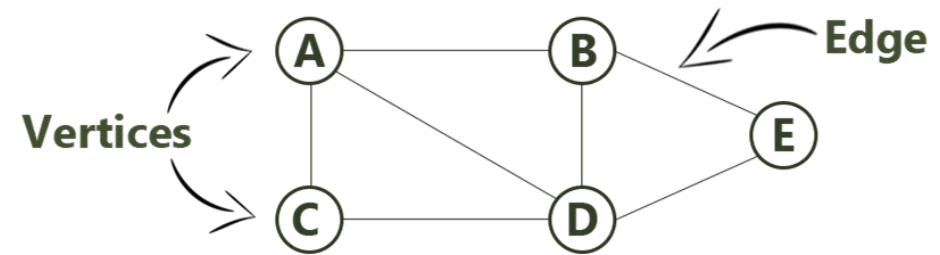


Non-Linear Data Structure Graph



Graphs

- ▶ What is Graph?
- ▶ Representation of Graph
 - ↳ Matrix representation of Graph
 - ↳ Linked List representation of Graph
- ▶ Elementary Graph Operations
 - ↳ Breadth First Search (BFS)
 - ↳ Depth First Search (DFS)
 - ↳ Spanning Trees
 - ↳ Minimal Spanning Trees
 - ↳ Shortest Path

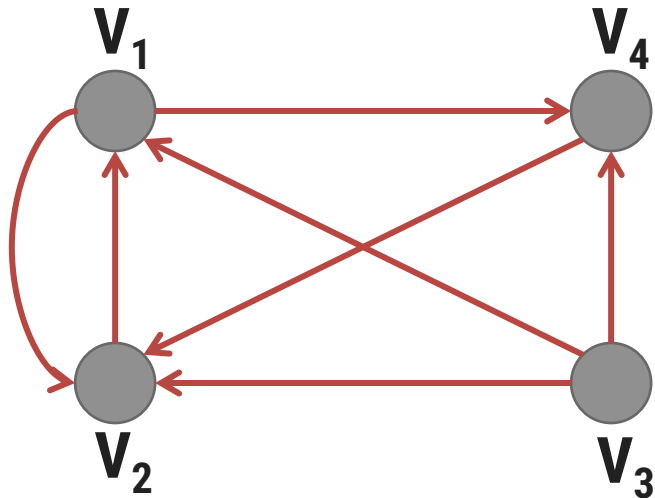
Adjacency matrix

- ▶ A **diagrammatic representation** of a **graph** may have limited usefulness. However such a representation **is not feasible** when number of **nodes** and **edges** in a graph **is large**
- ▶ It is easy to store and manipulate matrices and hence the graphs represented by them in the computer
- ▶ Let $G = (V, E)$ be a simple **diagraph** in which $V = \{v_1, v_2, \dots, v_n\}$ and the **nodes** are assumed to be **ordered** from v_1 to v_n
- ▶ An $n \times n$ matrix **A** is called **Adjacency matrix** of the graph G whose **elements** are a_{ij} are given by

$$a_{ij} = \begin{cases} 1 & \text{if } (v_i, v_j) \in E \\ 0 & \text{otherwise} \end{cases}$$

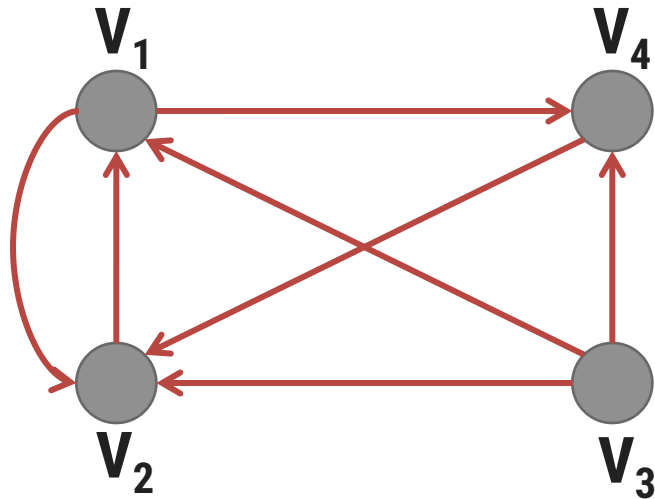
Adjacency matrix

- ▶ An **element** of the adjacency matrix is either **0** or **1**
- ▶ Any **matrix** whose **elements are either 0 or 1** is called **bit matrix** or **Boolean matrix**
- ▶ For a given graph $G = (V, E)$, an **adjacency matrix** depends upon the ordering of the elements of V
- ▶ For different ordering of the elements of V we get different adjacency matrices.



$$A = \begin{matrix} & \begin{matrix} V_1 & V_2 & V_3 & V_4 \end{matrix} \\ \begin{matrix} V_1 \\ V_2 \\ V_3 \\ V_4 \end{matrix} & \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \end{matrix}$$

Adjacency matrix



A =

$$\begin{matrix} & \begin{matrix} V_1 & V_2 & V_3 & V_4 \end{matrix} \\ \begin{matrix} V_1 \\ V_2 \\ V_3 \\ V_4 \end{matrix} & \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \end{matrix}$$

- ▶ The **number of elements** in the **i^{th} row** whose **value is 1** is equal to the **out-degree** of node **V_i**
- ▶ The **number of elements** in the **j^{th} column** whose **value is 1** is equal to the **in-degree** of node **V_j**
- ▶ For a **NULL graph** which consist of only n nodes but no edges, the **adjacency matrix** has **all its elements 0**. i.e. the adjacency matrix is the NULL matrix

Power of Adjacency matrix

$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

$$\mathbf{A}^2 = \mathbf{A} \times \mathbf{A} = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 2 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix}$$

$$\mathbf{A}^3 = \begin{pmatrix} 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 2 & 2 & 0 & 1 \\ 0 & 1 & 0 & 1 \end{pmatrix}$$

$$\mathbf{A}^4 = \begin{pmatrix} 1 & 2 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 2 & 3 & 0 & 2 \\ 1 & 1 & 0 & 0 \end{pmatrix}$$

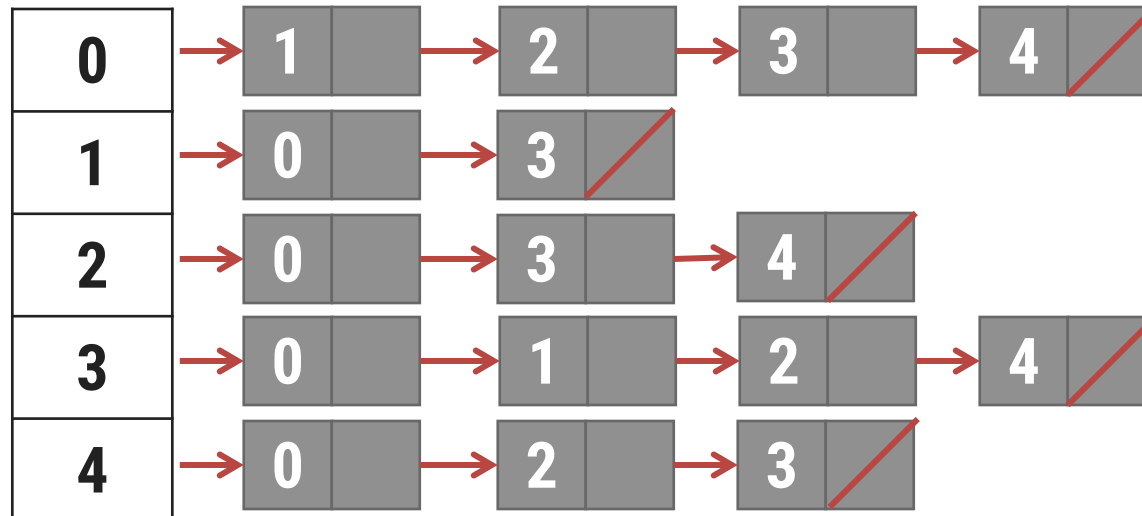
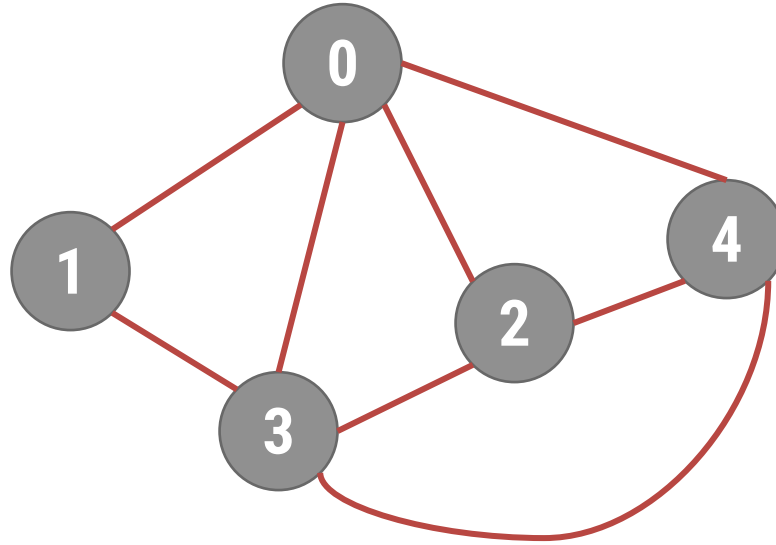
- ▶ Entry of **1** in **i^{th}** row and **j^{th}** column of **$\mathbf{A}$** shows existence of an **edge (V_i, V_j)** , that is a **path of length 1**
- ▶ Entry in **$\mathbf{A}^2$** shows **no of different paths** of **exactly length 2** from node **V_i** to **V_j**
- ▶ Entry in **$\mathbf{A}^3$** shows **no of different paths** of **exactly length 3** from node **V_i** to **V_j**

Path matrix or reachability matrix

- ▶ Let $G = (V, E)$ be a simple diagraph which contains **n nodes** that are assumed to be ordered.
- ▶ A **n x n** matrix **P** is called **path matrix** whose elements are given by

$$P_{ij} = \begin{cases} 1, & \text{if there exists path from node } V_i \text{ to } V_j \\ 0, & \text{otherwise} \end{cases}$$

Adjacency List Representation

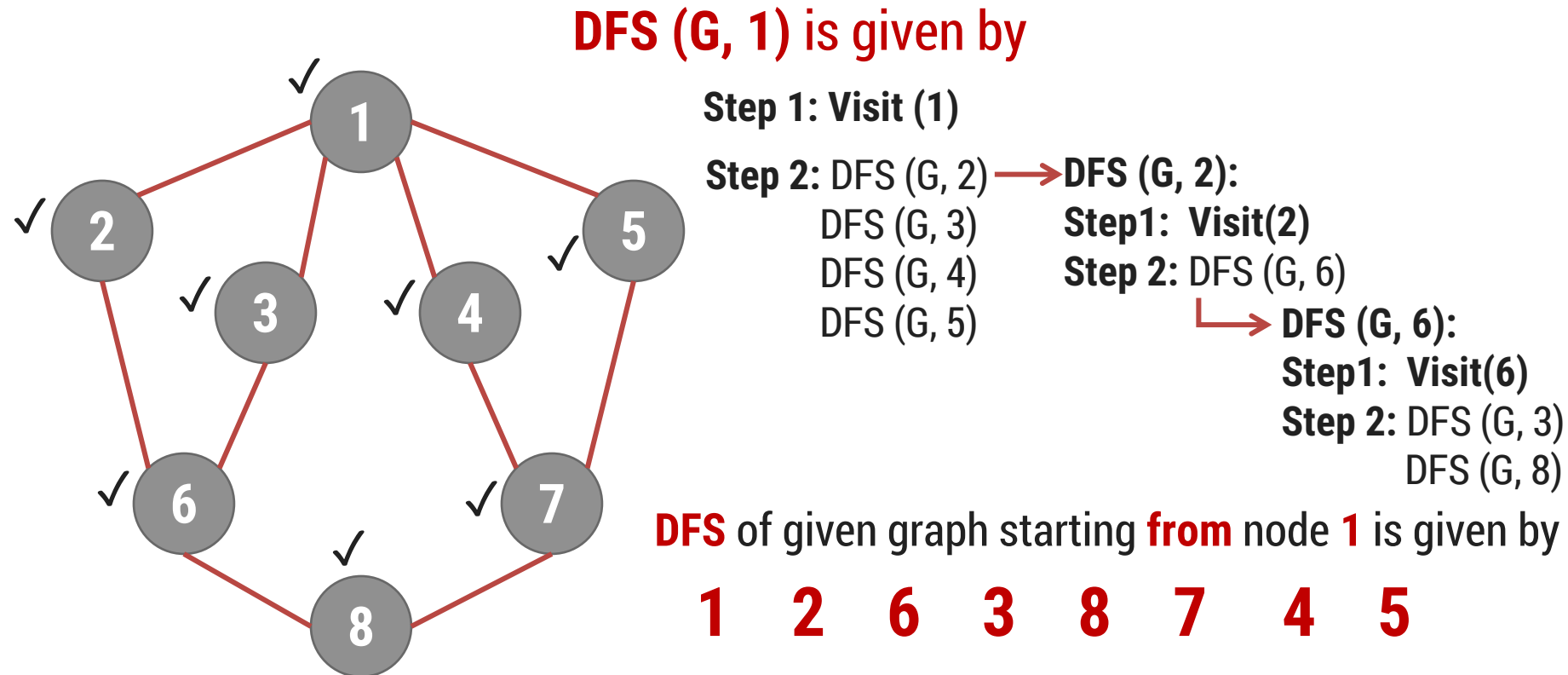


Graph Traversal

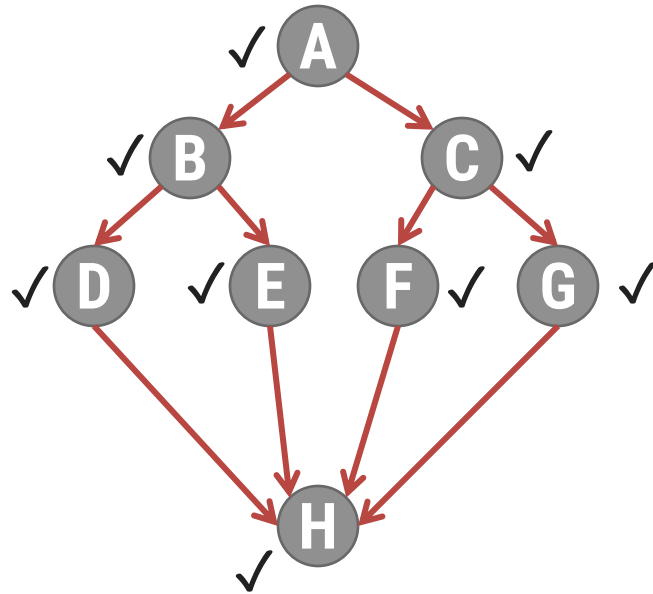
- ▶ Two Commonly used Traversal Techniques are
 - Depth First Search (DFS)
 - Breadth First Search (BFS)

Depth First Search (DFS)

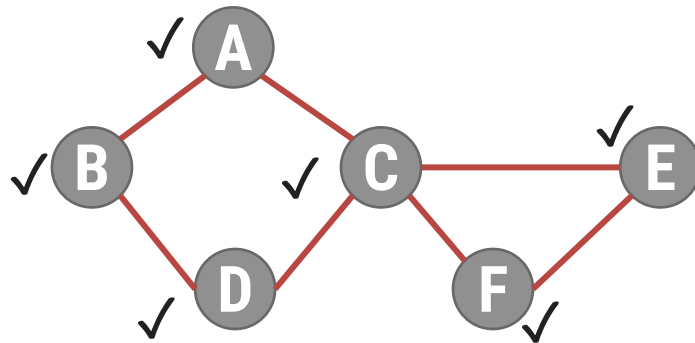
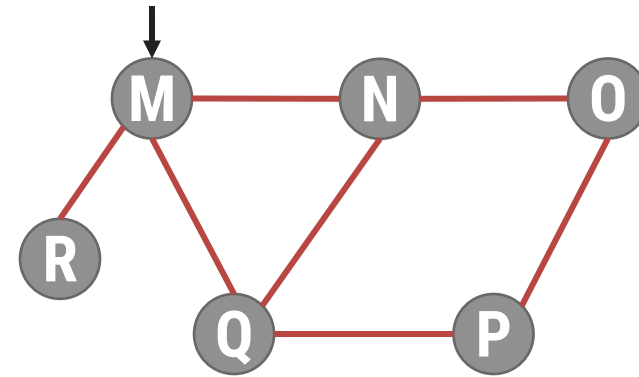
- ▶ It is like preorder traversal of tree
- ▶ Traversal can start from any vertex V_i
- ▶ V_i is visited and then all vertices adjacent to V_i are traversed recursively using DFS



Depth First Search (DFS)



A B D H E C F G

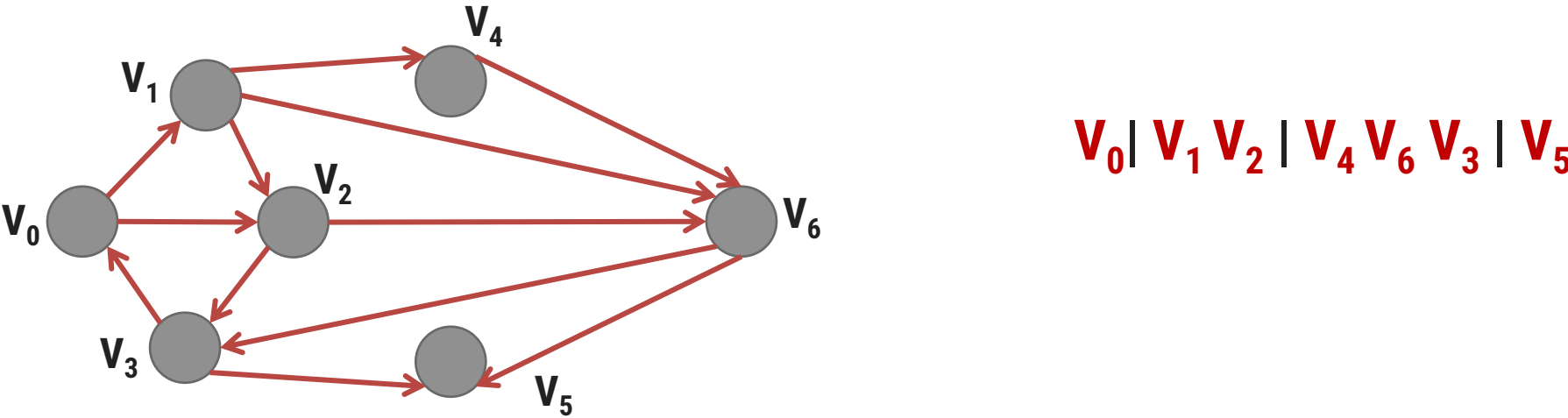
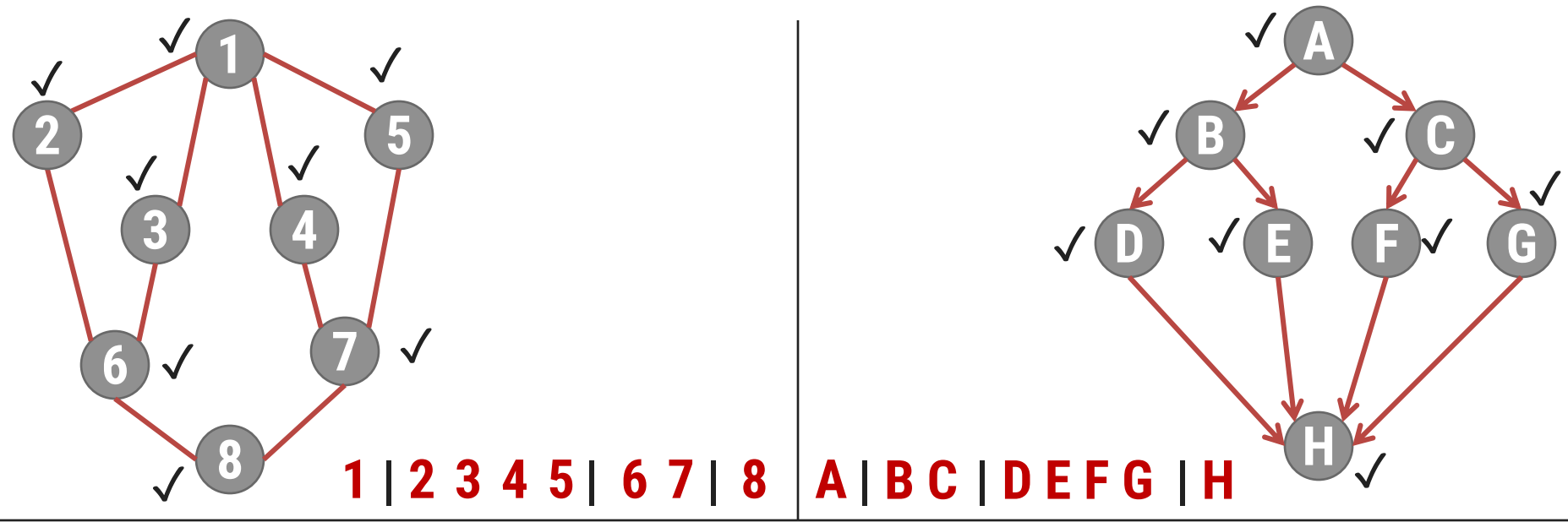


A B D C F E

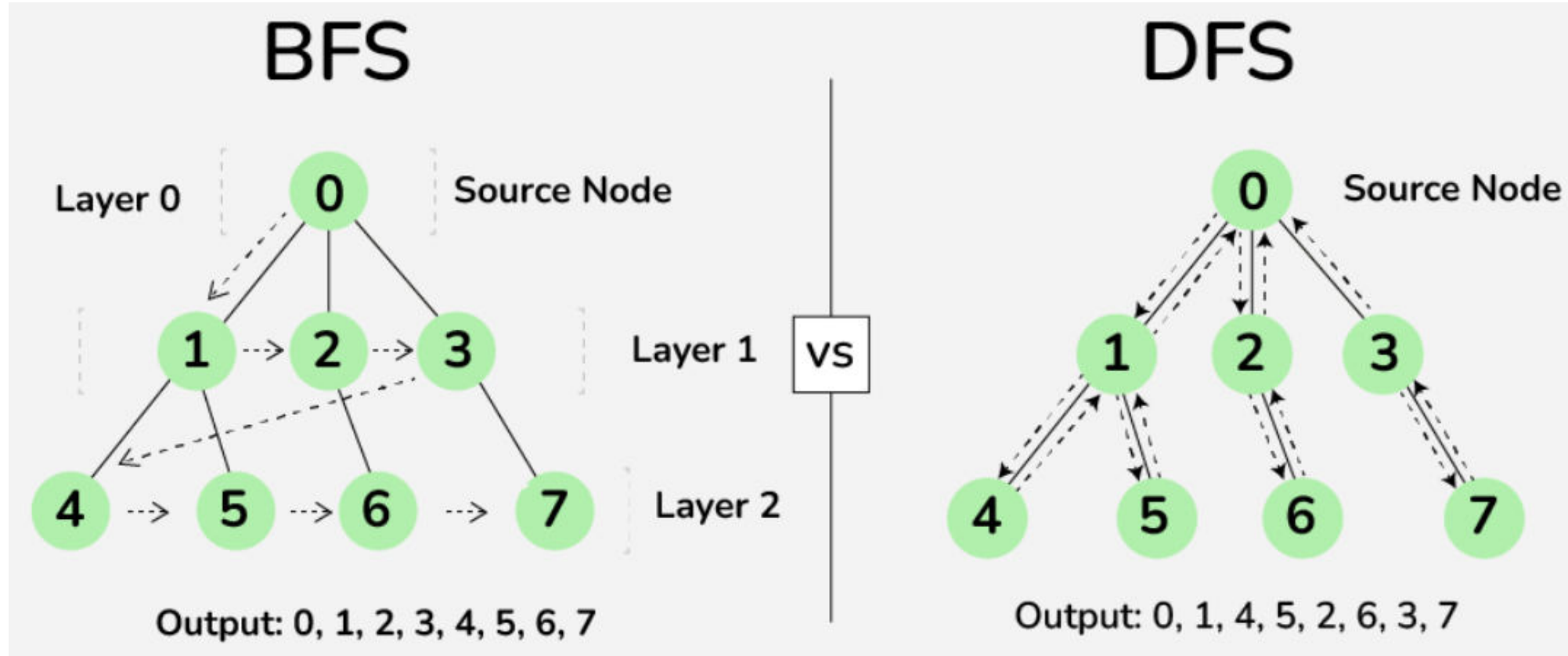
Breadth First Search (BFS)

- ▶ This method **starts** from vertex V_0
- ▶ V_0 is marked as **visited**. All **vertices adjacent to V_0** are **visited next**
- ▶ Let vertices adjacent to V_0 are V_1, V_2, V_2, V_4
- ▶ V_1, V_2, V_3 and V_4 are marked visited
- ▶ All unvisited vertices adjacent to V_1, V_2, V_3, V_4 are visited next
- ▶ The method **continuous until all vertices** are **visited**
- ▶ The algorithm for BFS has to maintain a list of vertices which have been visited but not explored for adjacent vertices
- ▶ The vertices which have been visited but not explored for adjacent vertices can be stored in **queue**

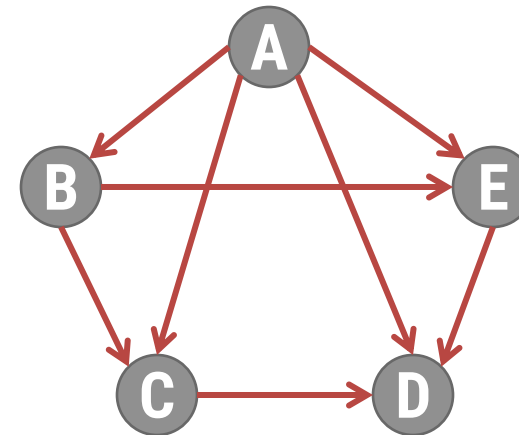
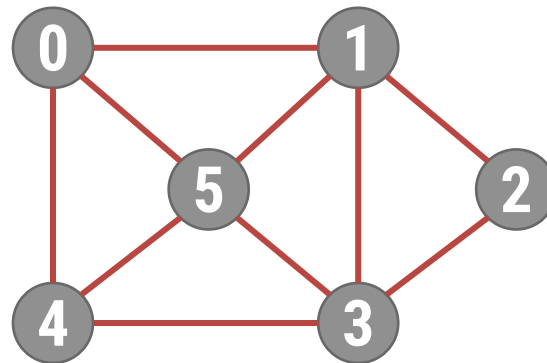
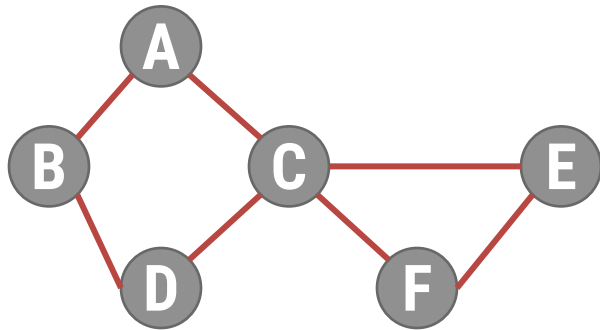
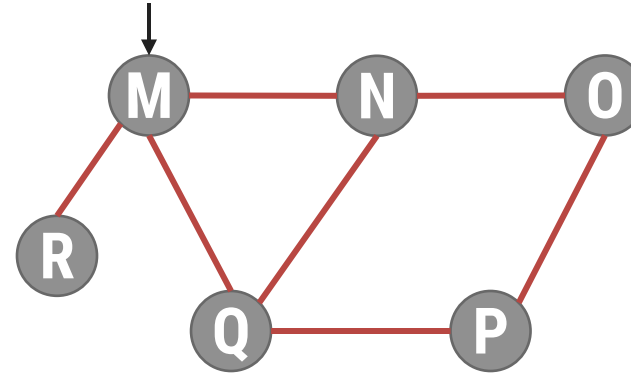
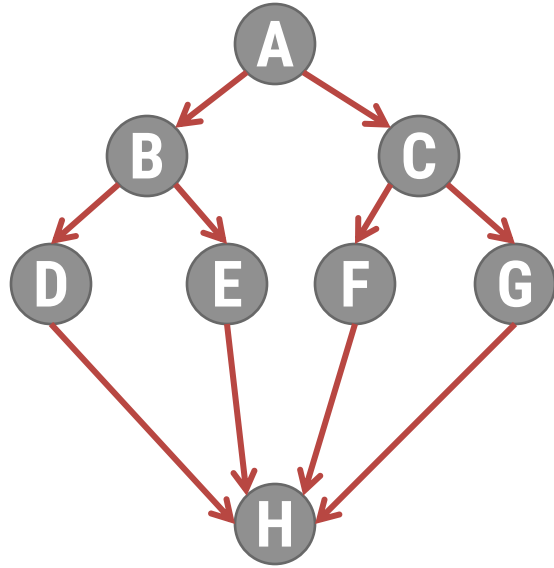
Breadth First Search (BFS)



Difference



Write DFS & BFS of following Graphs



Procedure : DFS (vertex V)

- ▶ This procedure **traverse the graph G in DFS** manner.
- ▶ V is a starting vertex to be explored.
- ▶ Visited[] is an array which tells you whether particular vertex is visited or not.
- ▶ W is a adjacent node of vertex V.
- ▶ S is a Stack, PUSH and POP are functions to insert and remove from stack respectively.

Procedure : DFS (vertex V)

1. [Initialize TOP and Visited]

visited[] $\leftarrow$ 0

TOP $\leftarrow$ 0

2. [Push vertex into stack]

PUSH (V)

3. [Repeat while stack is not Empty]

while stack is not empty

 v $\leftarrow$ POP()

 if visited[v] == 0

 visited [v] $\leftarrow$ 1

 for all u adjacent to v

 if visited [u] == 0

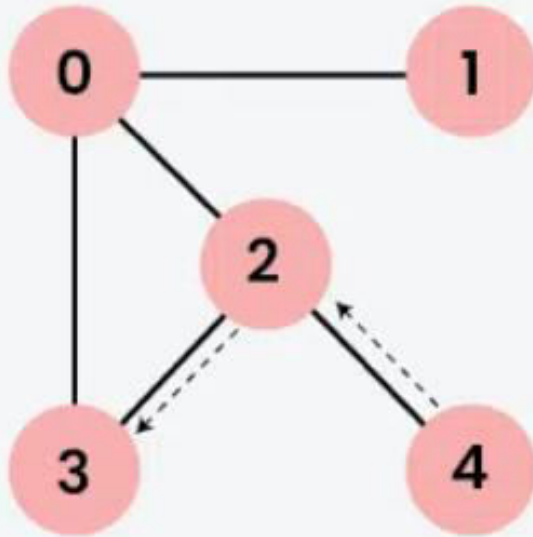
 then PUSH (u)

 end for

 end if

05
Step

Now, Node 3 at the top of the stack, so visit node 3 and pop it from the stack and put all of its adjacent nodes which are not visited in the stack.



Visited

DFS

Stack

0	1	2	3	4
T	T	T	T	T
0	1	2	4	3

Now, Stack becomes empty, which means we have visited all the nodes and our DFS traversal ends.

Complexity Analysis

- ▶ **Time complexity:** $O(V + E)$, Note that the time complexity is same here because we visit every vertex at most once and every edge is traversed at most once (in directed) and twice in undirected.
- ▶ **Auxiliary Space:** $O(V + E)$, since an extra visited array of size V is required, And stack size for recursive calls to DFSRec function.

Procedure : BFS (vertex V)

- ▶ This procedure **traverse the graph G in BFS** manner
- ▶ **V** is a **starting vertex** to be explored
- ▶ Q is a queue
- ▶ visited[] is an array which tells you whether particular vertex is visited or not
- ▶ W is a adjacent node f vertex V.

Procedure : BFS (vertex V)

1. [Initialize Queue & Visited]

$\text{visited[]} \leftarrow 0$

$F \leftarrow R \leftarrow 0$

2. [Marks visited of V as 1]

$\text{visited}[v] \leftarrow 1$

3. [Add vertex v to Q]

$\text{EnQueue}(V)$

4. [Repeat while Q is not Empty]

Repeat while Q is not empty

$v \leftarrow \text{DeQueue}()$

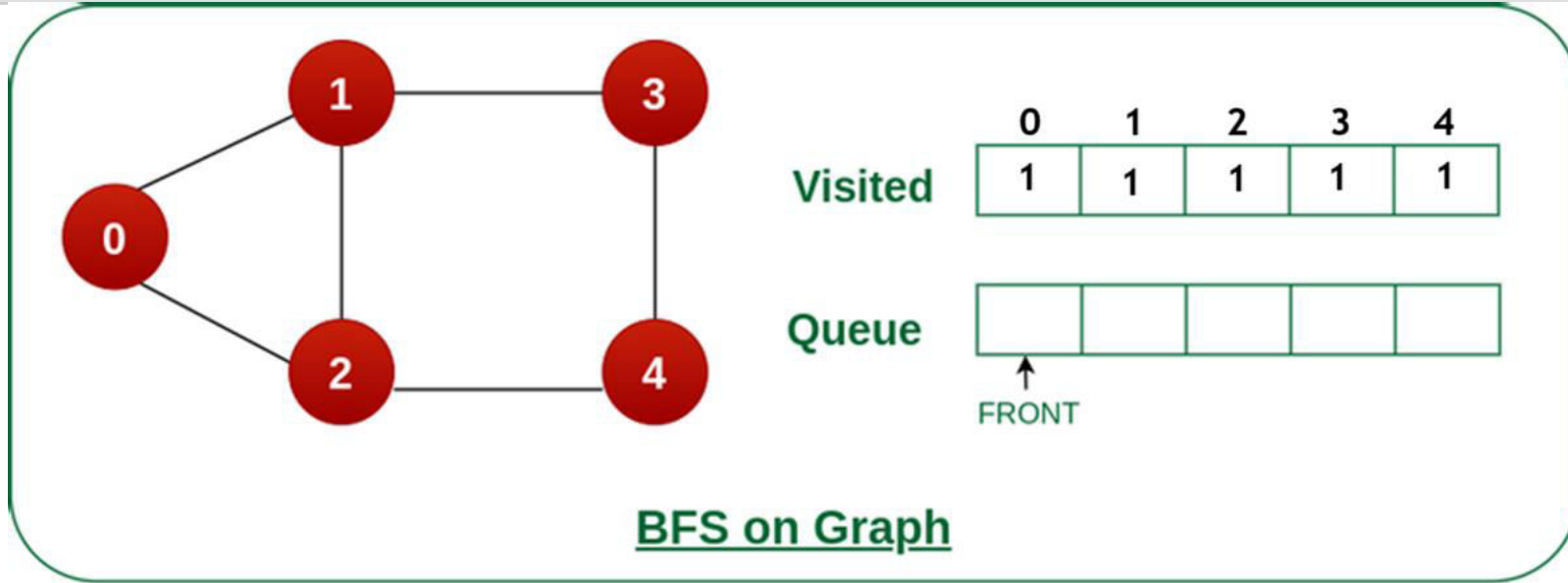
For all vertices u adjacent to v

If $\text{visited}[u]$ is 0

Then $\text{visited}[u] \leftarrow 1$

$\text{EnQueue}(u)$

BFS



Complexity Analysis of Breadth-First Search (BFS) Algorithm:

Time Complexity of BFS Algorithm: $O(V + E)$

- ▶ BFS explores all the vertices and edges in the graph. In the worst case, it visits every vertex and edge once. Therefore, the time complexity of BFS is $O(V + E)$, where V and E are the number of vertices and edges in the given graph.

Auxiliary Space of BFS Algorithm: $O(V)$

- ▶ BFS uses a queue to keep track of the vertices that need to be visited. In the worst case, the queue can contain all the vertices in the graph. Therefore, the space complexity of BFS is $O(V)$.

Applications of BFS in Graphs:

- ▶ **Shortest Path Finding:** BFS can be used to find the shortest path between two nodes in an unweighted graph. By keeping track of the parent of each node during the traversal, the shortest path can be reconstructed.
- ▶ **Cycle Detection:** BFS can be used to detect cycles in a graph. If a node is visited twice during the traversal, it indicates the presence of a cycle.
- ▶ **Connected Components:** BFS can be used to identify connected components in a graph. Each connected component is a set of nodes that can be reached from each other.
- ▶ **Topological Sorting:** BFS can be used to perform topological sorting on a directed acyclic graph (DAG). Topological sorting arranges the nodes in a linear order such that for any edge (u, v) , u appears before v in the order.
- ▶ **Level Order Traversal of Binary Trees:** BFS can be used to perform a level order traversal of a binary tree. This traversal visits all nodes at the same level before moving to the next level.
- ▶ **Network Routing:** BFS can be used to find the shortest path between two nodes in a network, making it useful for routing data packets in network protocols.

Minimum Spanning Tree

Spanning tree

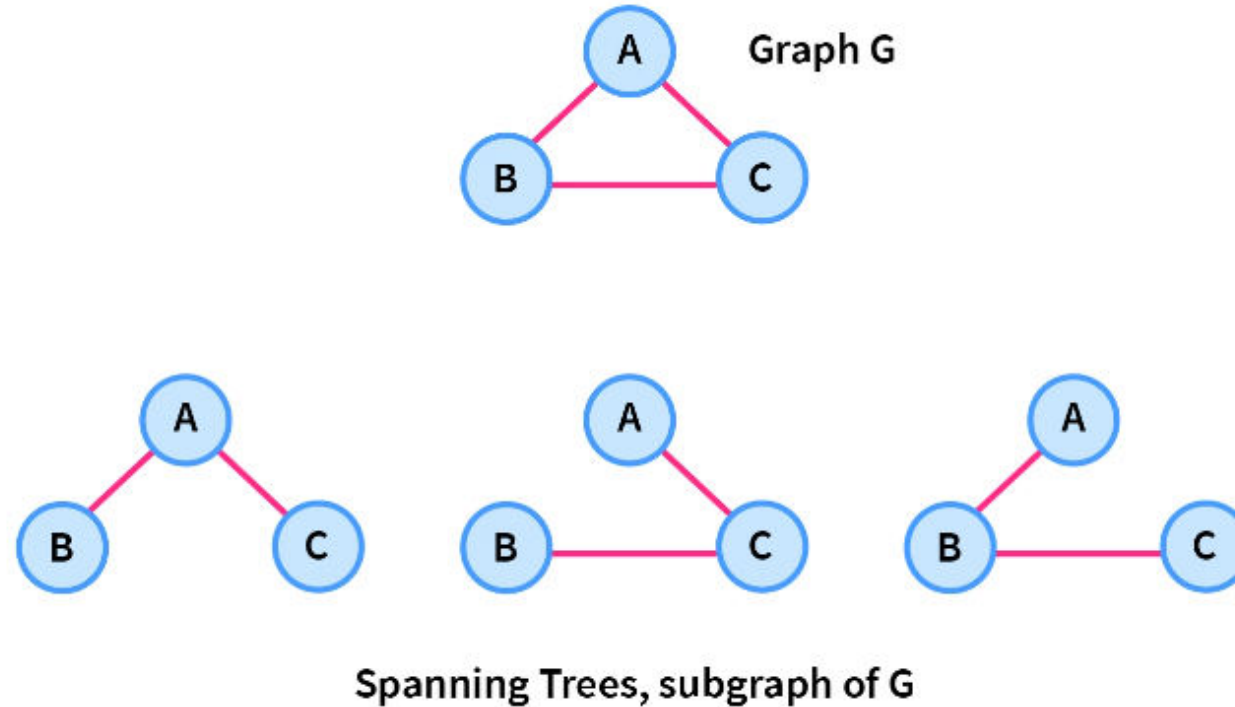
- ▶ A spanning tree is a sub-graph that connects all the vertices of a graph with the minimum possible number of edges. It may or may not be weighted and does not have cycles.
- ▶ Let us understand what is spanning tree with an interesting example.
- ▶ Consider a situation where a cable television network company is laying the cable to a new neighborhood. If the company is constrained to bury the cable only along certain paths for example along the roads then the problem is to find the minimum amount of cable that the company requires to complete the wiring of the television network

Spanning tree

- ▶ A Spanning tree always contains $n-1$ edges, where n is the total number of vertices in the graph G .
- ▶ The total number of spanning trees that a complete graph of n vertices can have is $n(n-2)$.
- ▶ We can construct a spanning tree by removing atmost $e-n+1$ edges from a complete graph G , where e is the number of edges and n is the number of vertices in graph G .

Spanning tree

- ▶ A complete undirected graph G can have a maximum n^{n-2} number of spanning trees, where n is the number of nodes in a given graph G . Let us Consider a complete graph G with 3 vertices, then the total number of spanning trees this graph can have is $3^{(3-2)}=3$ which are shown in the image below.

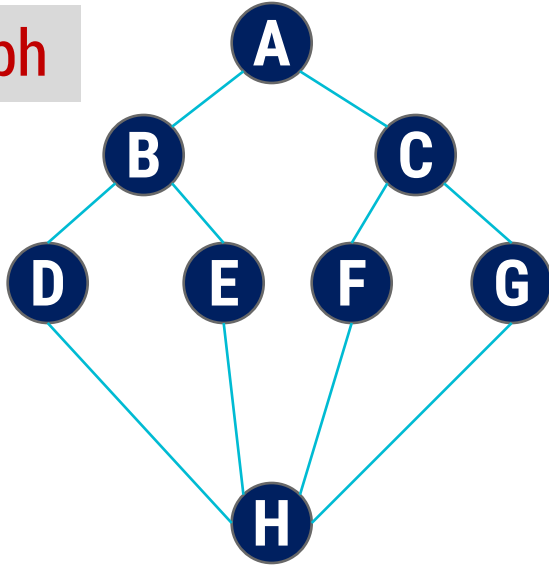


Introduction to Minimum Spanning Tree (MST)

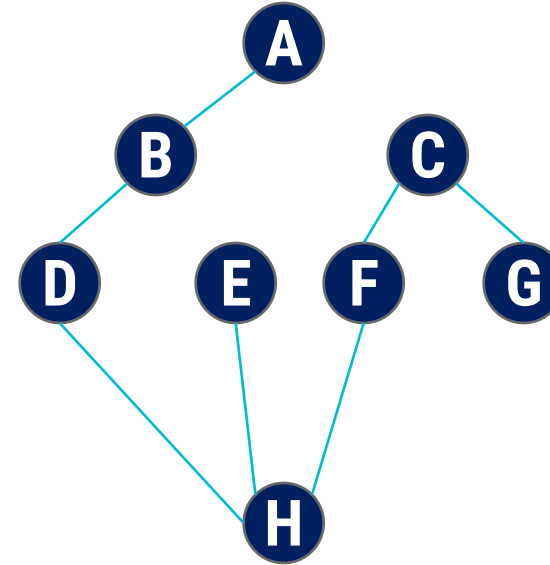
- ▶ Let $G = \langle N, A \rangle$ be a **connected, undirected graph** where,
 1. N is the set of nodes and
 2. A is the set of edges.
- ▶ Each edge has a given **positive length or weight**.
- ▶ A spanning tree of a graph G is a **sub-graph** which is basically a tree and it contains all the vertices of G but **does not contain cycle**.
- ▶ A minimum spanning tree (MST) of a **weighted connected graph** G is a spanning tree with **minimum or smallest weight of edges**.
- ▶ Two Algorithms for **constructing** minimum spanning tree are,
 1. **Kruskal's Algorithm**
 2. **Prim's Algorithm**

Spanning Tree Examples

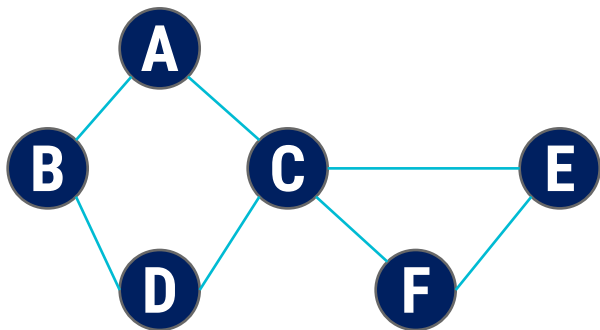
Graph



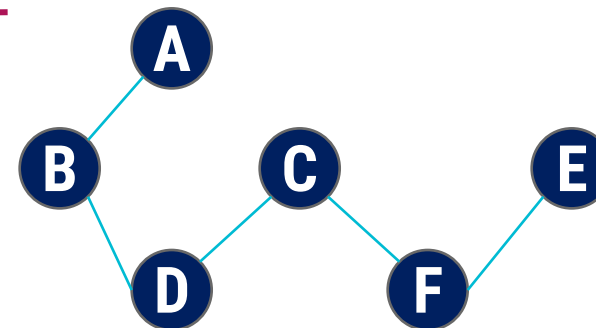
Spanning Tree



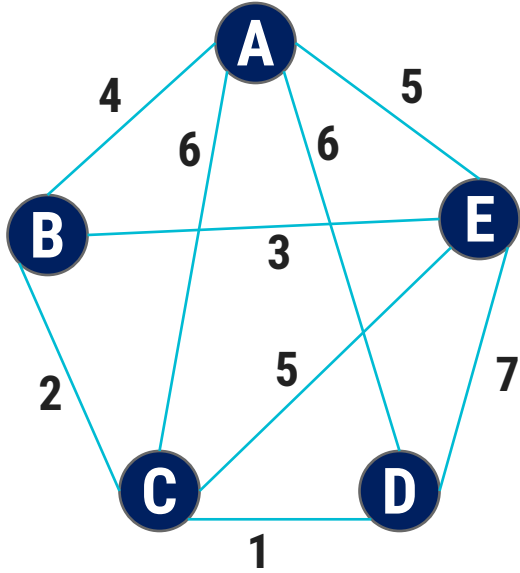
Graph



Spanning Tree



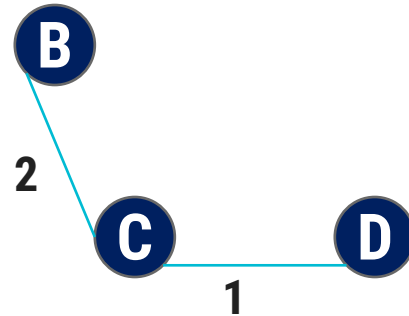
Kruskal's Algorithm for MST – Example 1



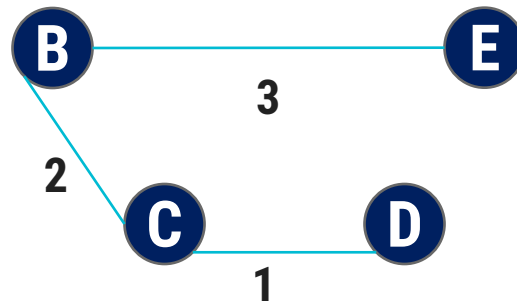
Step 1: Taking min edge (C,D)



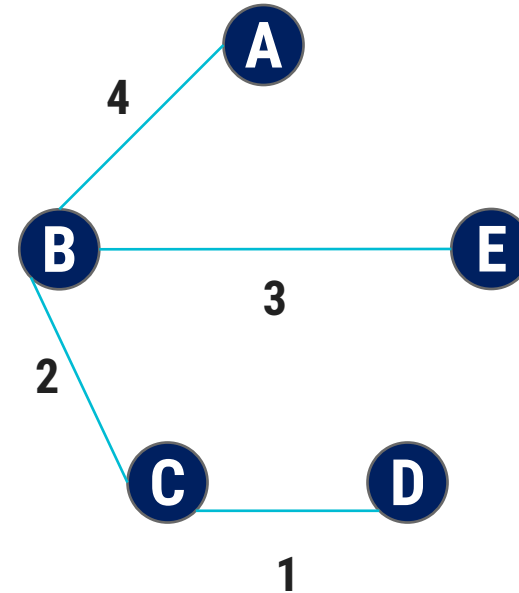
Step 2: Taking next min edge (B,C)



Step 3: Taking next min edge (B,E)



Step 4: Taking next min edge (A,B)

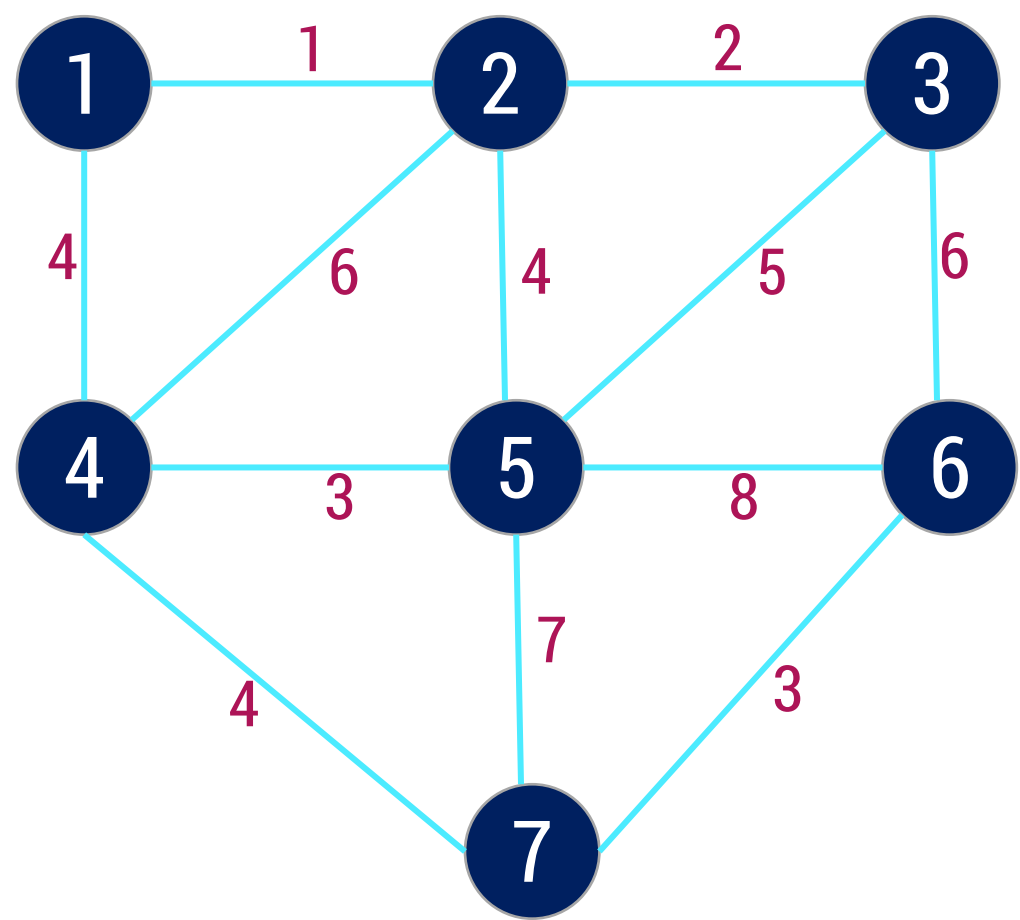


So, we obtained a minimum spanning tree of cost:
 $4 + 2 + 1 + 3 = 10$

Kruskal's Algorithm for MST – Example 2

Step:1

Sort the edges in increasing order of their weight.

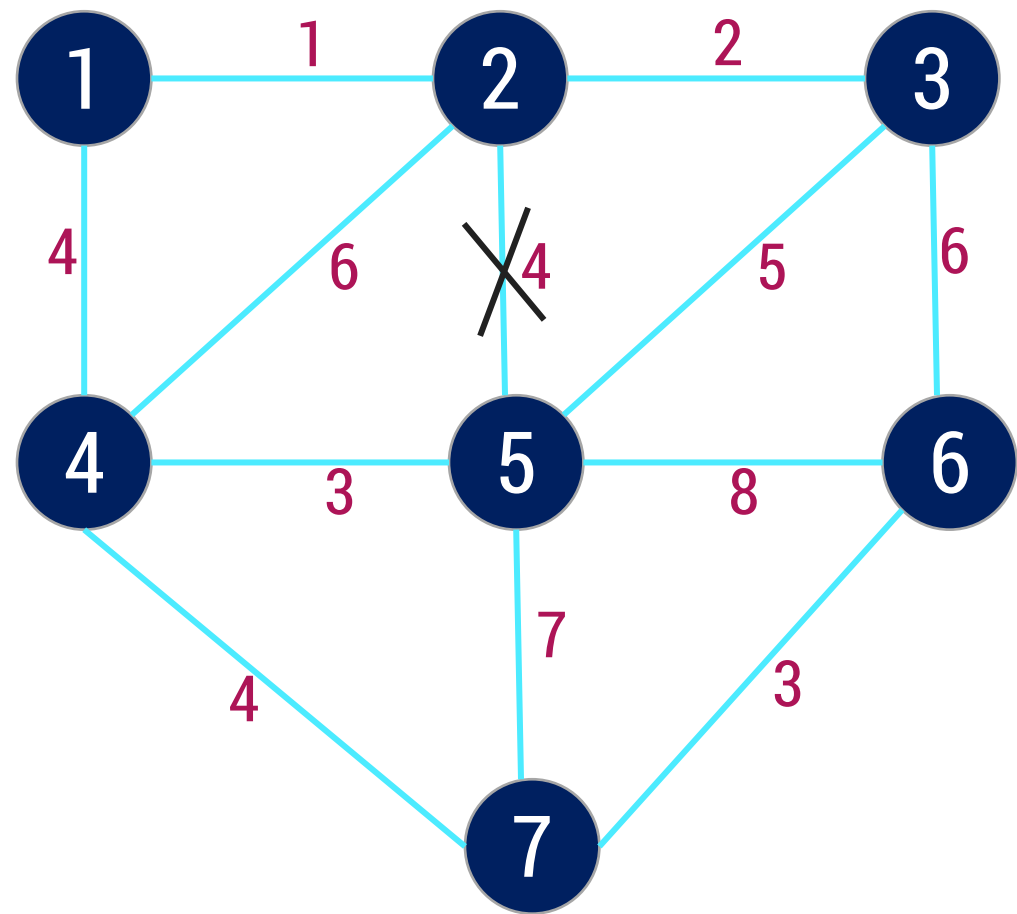


Edges	Weight	
{1, 2}	1	
{2, 3}	2	
{4, 5}	3	
{6, 7}	3	
{1, 4}	4	
{2, 5}	4	
{4, 7}	4	
{3, 5}	5	
{2, 4}	6	
{3, 6}	6	
{5, 7}	7	
{5, 6}	8	

Kruskal's Algorithm for MST – Example 2

Step:2

Select the minimum weight edge but no cycle.

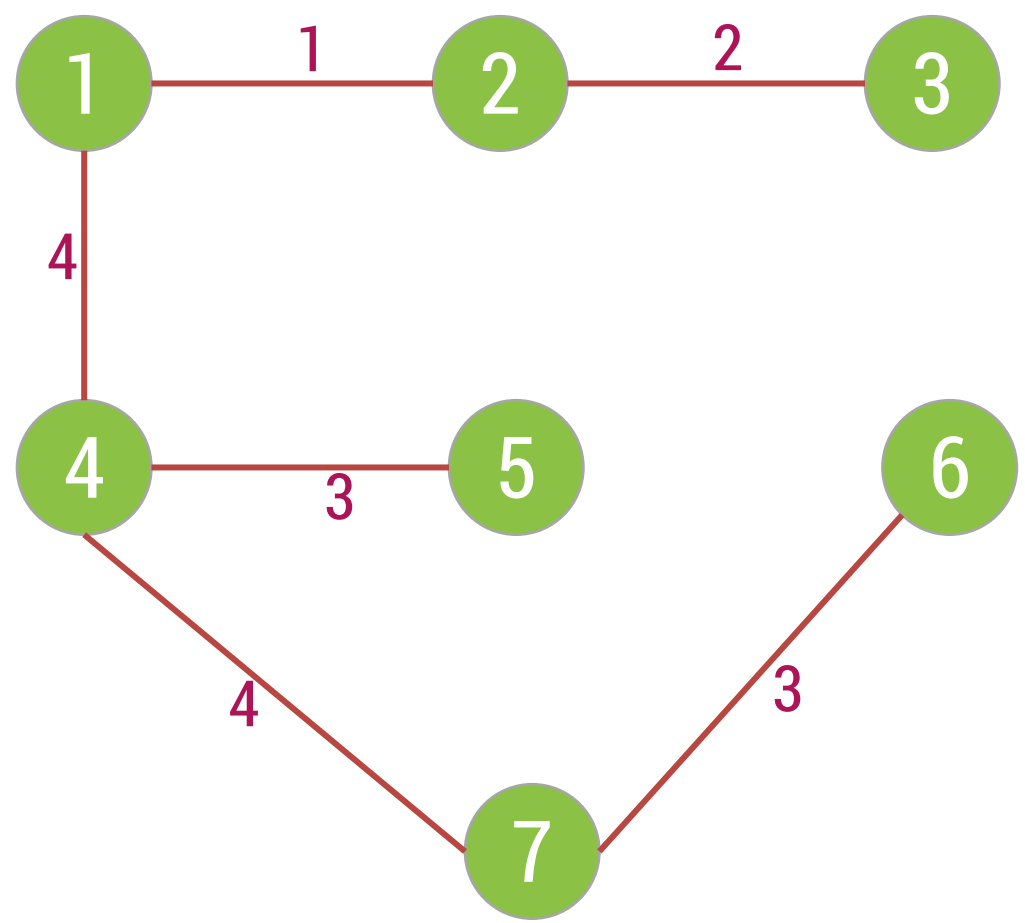


Edges	Weight	
{1, 2}	1	✓
{2, 3}	2	✓
{4, 5}	3	✓
{6, 7}	3	✓
{1, 4}	4	✓
{2, 5}	4	
{4, 7}	4	✓
{3, 5}	5	
{2, 4}	6	
{3, 6}	6	
{5, 7}	7	
{5, 6}	8	

Kruskal's Algorithm for MST – Example 2

Step:3

The minimum spanning tree for the given graph.



Edges	Weight	
{1, 2}	1	✓
{2, 3}	2	✓
{4, 5}	3	✓
{6, 7}	3	✓
{1, 4}	4	✓
{4, 7}	4	✓

Total Cost = 17

Kruskal's Algorithm – Example 2

Step	Edges considered - {u, v}	Connected Components
		
		
		
		
		
		

Edges	Weight

Total Cost = 17

Kruskal's Algorithm for MST

MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

```
function MakeSet( $x$ )
```

```
     $x.parent = x$ 
```

```
function Find( $x$ )
```

```
    if  $x.parent == x$ 
```

```
        return  $x$ 
```

```
    else
```

```
        return Find( $x.parent$ )
```

```
function Union( $x, y$ )
```

```
     $xRoot = Find(x)$ 
```

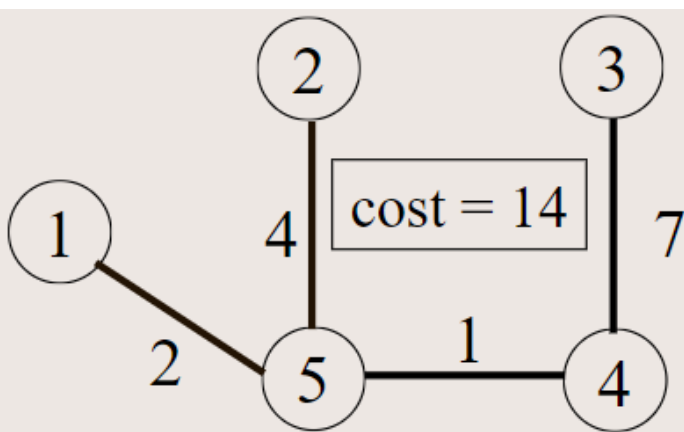
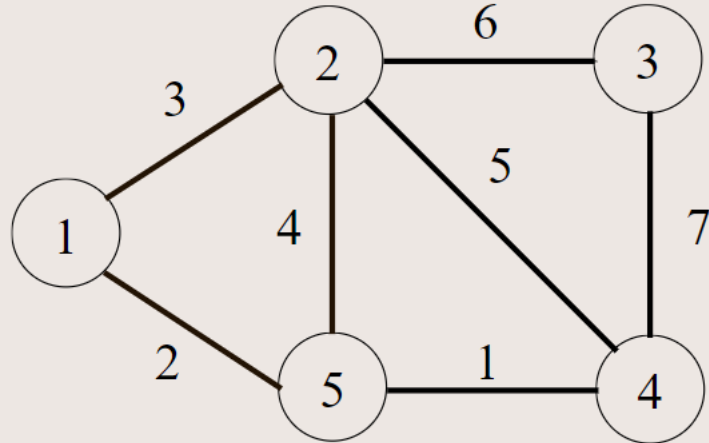
```
     $yRoot = Find(y)$ 
```

```
     $xRoot.parent = yRoot$ 
```

MAKE-SET(x) creates a new set whose only member (and thus representative) is x . Since the sets are disjoint, we require that x not already be in some other set.

FIND-SET(x) returns a pointer to the representative of the (unique) set containing x .

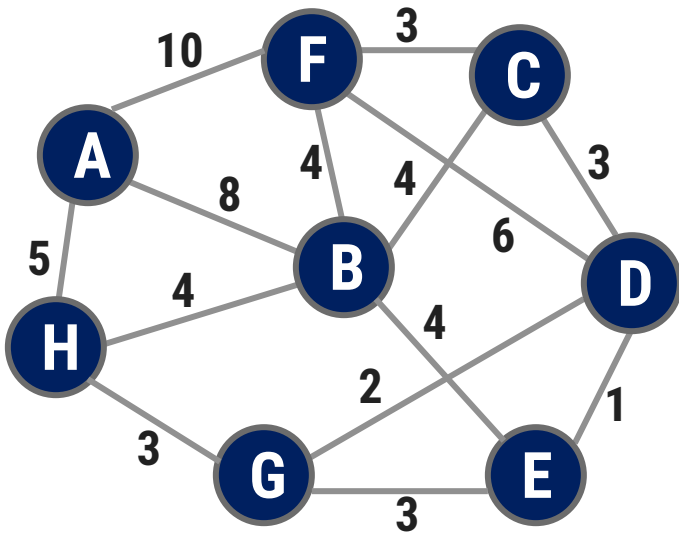
UNION(x,y) unites the dynamic sets that contain x and y



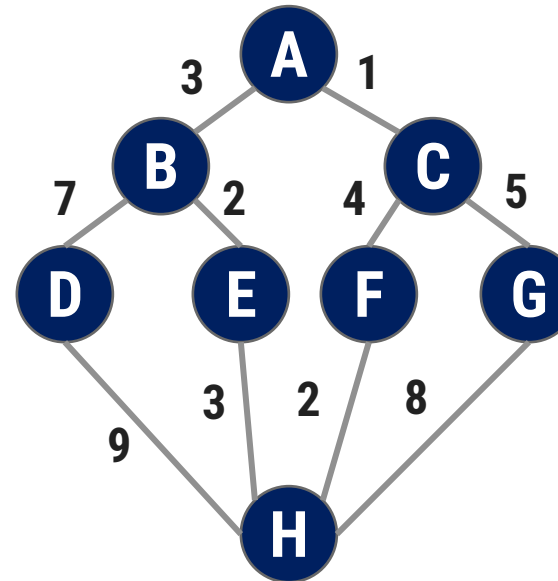
Exercises – Home Work

- Write the Kruskal's Algorithm to find out Minimum Spanning Tree. Apply the same and find MST for the graph given below.

1.



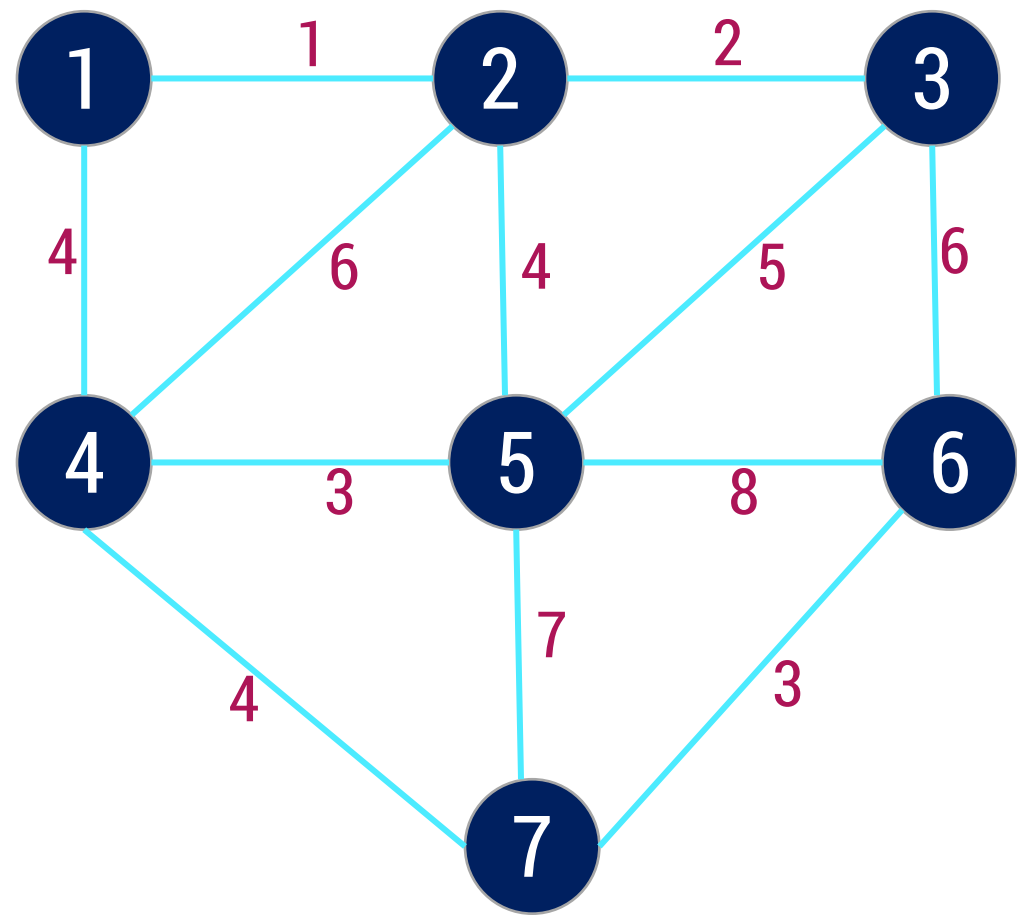
2.



Prim's Algorithm

- ▶ In Prim's algorithm, the minimum spanning tree grows in a natural way, starting from an arbitrary root.
- ▶ At each stage we add a new branch to the tree already constructed; the algorithm stops when all the nodes have been reached.
- ▶ The complexity for the Prim's algorithm is $\theta(n^2)$ where n is the total number of nodes in the graph G .

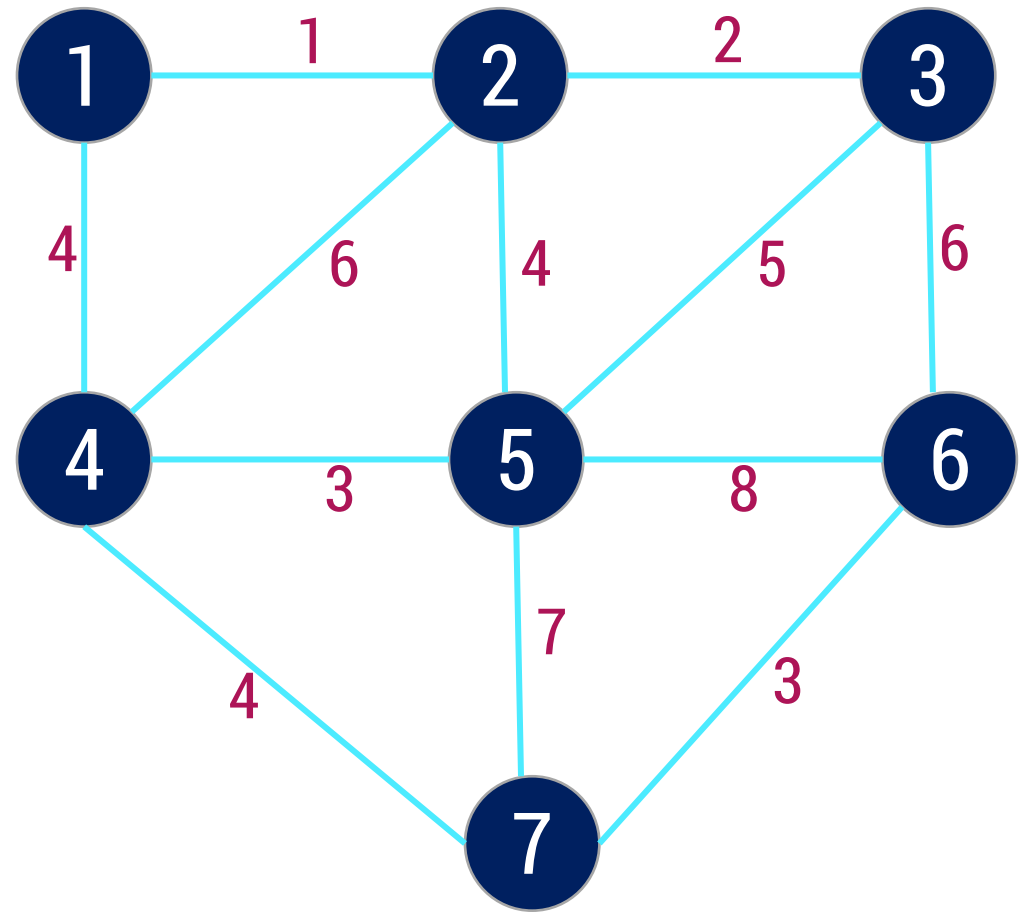
Prim's Algorithm for MST – Example 1



Step:1 Select an arbitrary node.

Node - Set B	Edges
1	

Prim's Algorithm for MST – Example 1

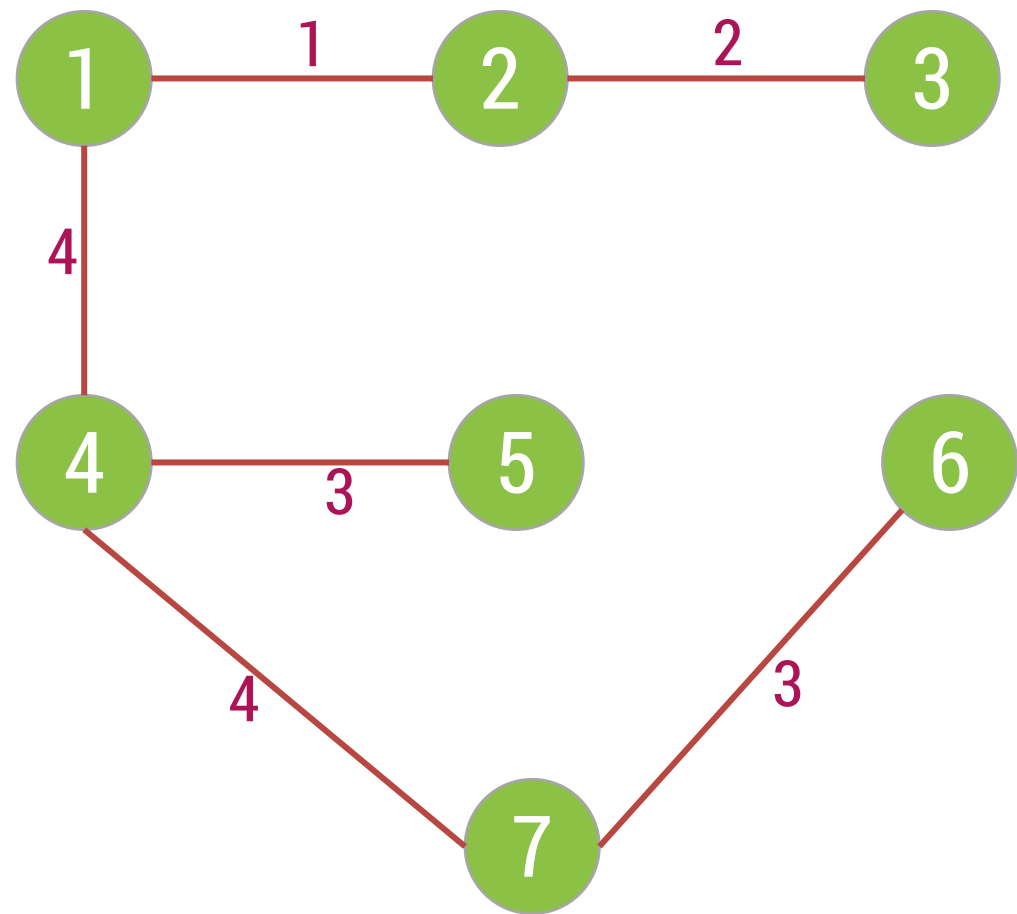


Step:2 Find an edge with minimum weight.

Node - Set B	Edges
1	{1, 2}, {1, 4}
1, 2	{1, 4} , {2, 3} {2, 4} , {2, 5}
1, 2, 3	{1,4}, {2,4} , {2,5} , {3,5}, {3,6}
1, 2, 3, 4	{2,4} {2,5} {3,5} {3,6} {4,5} {4,7}
1, 2, 3, 4, 5	{2,4} {2,5} {3,5} {3,6} {4,7} {5,6} {5,7}
1, 2, 3, 4, 5, 7	{2,4} {2,5} {3,5} {3,6} {5,6} {5,7} {6,7}

1, 2, 3, 4, 5, 6, 7

Prim's Algorithm for MST – Example 1



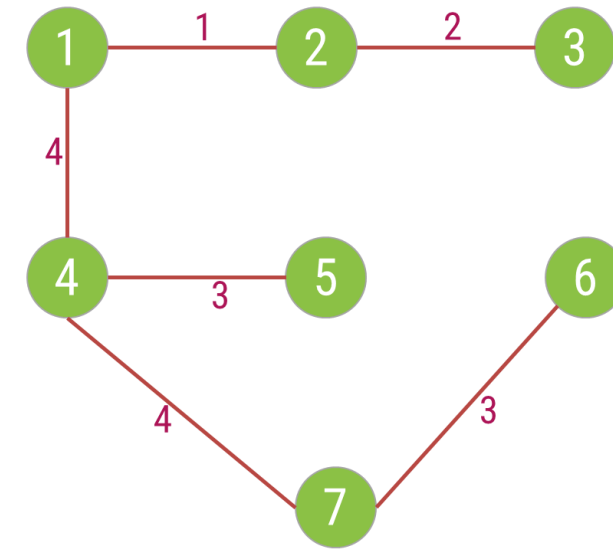
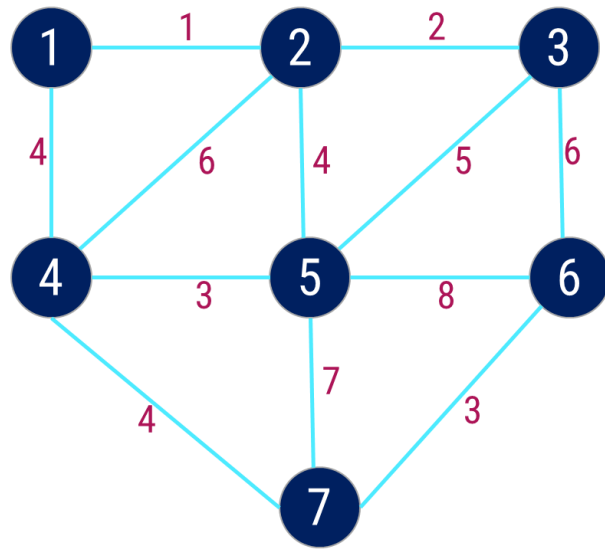
Step:3

The minimum spanning tree for the given graph.

Node	Edges
1	
1, 2	{1, 2}
1, 2, 3	{2, 3}
1, 2, 3, 4	{1, 4}
1, 2, 3, 4, 5	{4, 5}
1, 2, 3, 4, 5, 7	{4, 7}
1, 2, 3, 4, 5, 6, 7	{6, 7}

Total Cost = 17

Prim's Algorithm – Example 1



Cost = 17

Step	Edge Selected $\{u, v\}$	Set B	Edges Considered
Init.	-	$\{1\}$	--
1	$\{1, 2\}$	$\{1, 2\}$	$\{1, 2\}$ $\{1, 4\}$
2	$\{2, 3\}$	$\{1, 2, 3\}$	$\{1, 4\}$ $\{2, 3\}$ $\{2, 4\}$ $\{2, 5\}$
3	$\{1, 4\}$	$\{1, 2, 3, 4\}$	$\{1, 4\}$ $\{2, 4\}$ $\{2, 5\}$ $\{3, 5\}$ $\{3, 6\}$
4	$\{4, 5\}$	$\{1, 2, 3, 4, 5\}$	$\{2, 4\}$ $\{2, 5\}$ $\{3, 5\}$ $\{3, 6\}$ $\{4, 5\}$ $\{4, 7\}$
5	$\{4, 7\}$	$\{1, 2, 3, 4, 5, 7\}$	$\{2, 4\}$ $\{2, 5\}$ $\{3, 5\}$ $\{3, 6\}$ $\{4, 7\}$ $\{5, 6\}$ $\{5, 7\}$
6	$\{6, 7\}$	$\{1, 2, 3, 4, 5, 6, 7\}$	$\{2, 4\}$ $\{2, 5\}$ $\{3, 5\}$ $\{3, 6\}$ $\{5, 6\}$ $\{5, 7\}$ $\{6, 7\}$

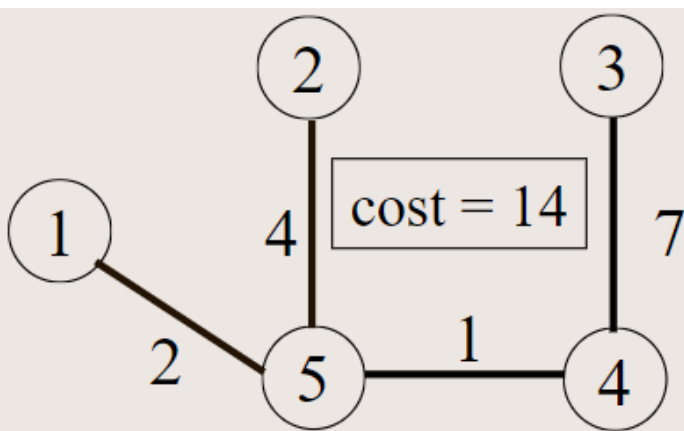
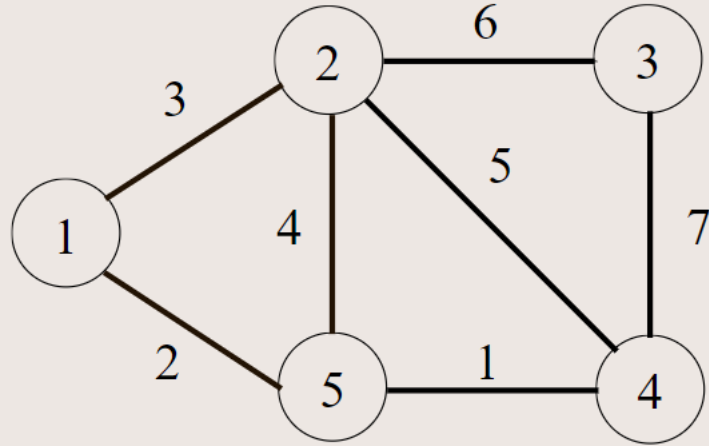
Prim's Algorithm

MST-PRIM(G, w, r)

```
1  for each  $u \in G.V$ 
2       $u.key = \infty$ 
3       $u.\pi = \text{NIL}$ 
4   $r.key = 0$ 
5   $Q = G.V$ 
6  while  $Q \neq \emptyset$ 
7       $u = \text{EXTRACT-MIN}(Q)$ 
8      for each  $v \in G.Adj[u]$ 
9          if  $v \in Q$  and  $w(u, v) < v.key$ 
10              $v.\pi = u$ 
11              $v.key = w(u, v)$ 
```

Alternatively

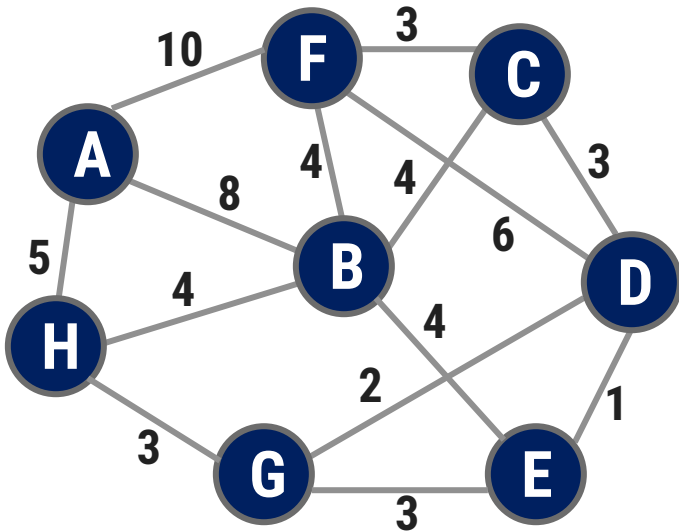
1. Create an array `parent[]` of size V to store indexes of parent nodes in MST.
2. Also create an array `key[]` to store key values of all vertices.
3. Initialize all key values as INFINITE.
4. Assign key value as 0 for the first vertex so that it is picked first.
6. Create a boolean array `mstSet[]` to represent the set of vertices included in MST.
7. While `mstSet` doesn't include all vertices
 6. Pick a vertex u which is not there in `mstSet` and has minimum key value.
 7. Include u to `mstSet`.
 8. Update the key value of all adjacent vertices of u according to the respective edge weights.
7. The parent array is the output array that is used to show the constructed MST.



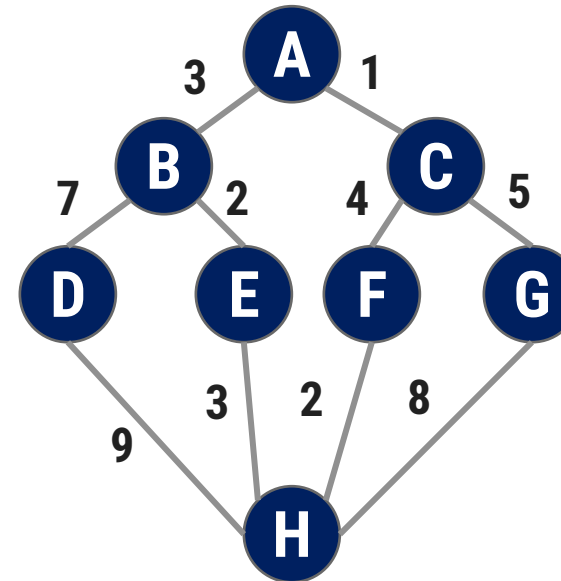
Exercises – Home Work

- Write the Prim's Algorithm to find out Minimum Spanning Tree. Apply the same and find MST for the graph given below.

1.



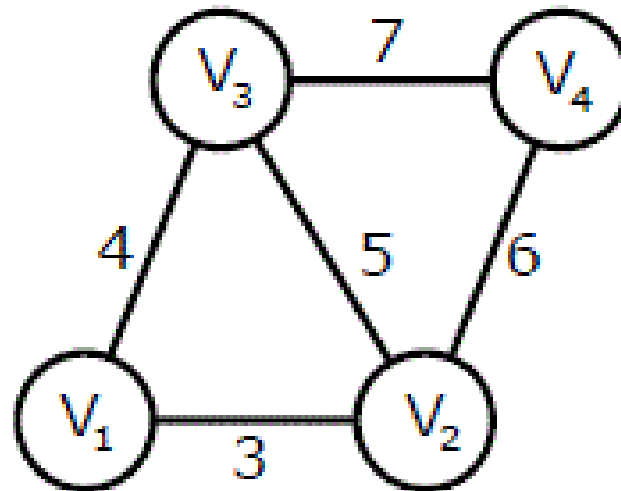
2.



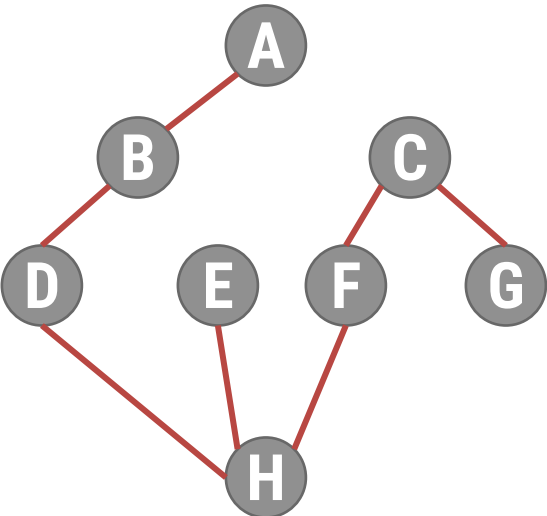
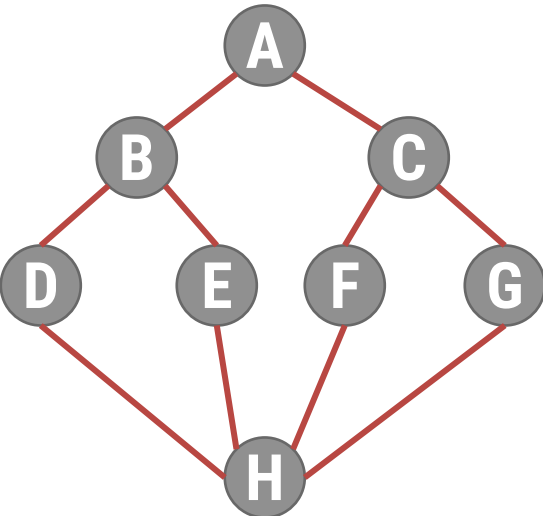
Exercise

- ▶ An undirected graph $G(V, E)$ contains n ($n > 2$) nodes named $v_1, v_2, \dots, v_n$. Two nodes v_i, v_j are connected if and only if $0 < |i - j| \leq 2$. Each edge (v_i, v_j) is assigned a weight $i + j$.

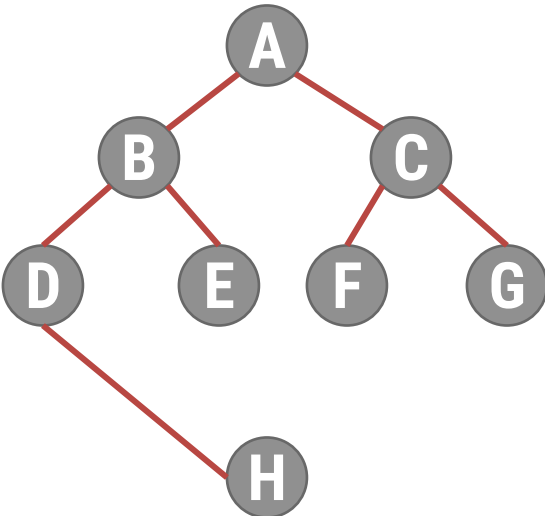
Create a graph for $n=4$.



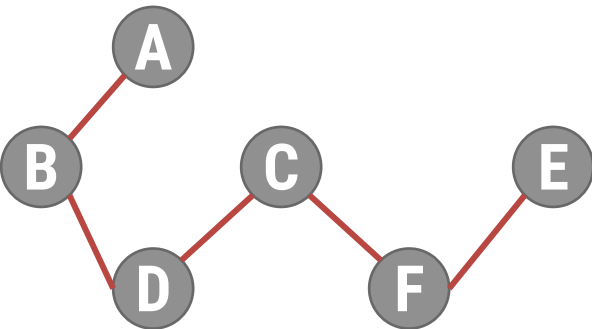
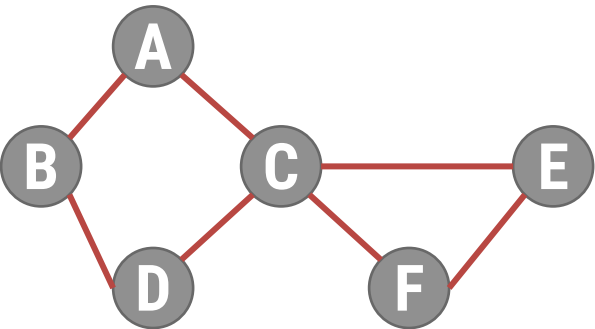
Construct Spanning Tree



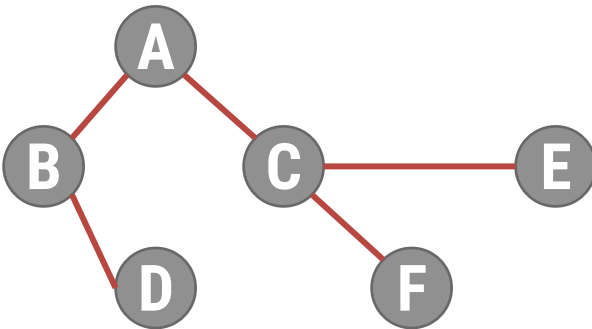
DFS Spanning Tree



BFS Spanning Tree



DFS Spanning Tree

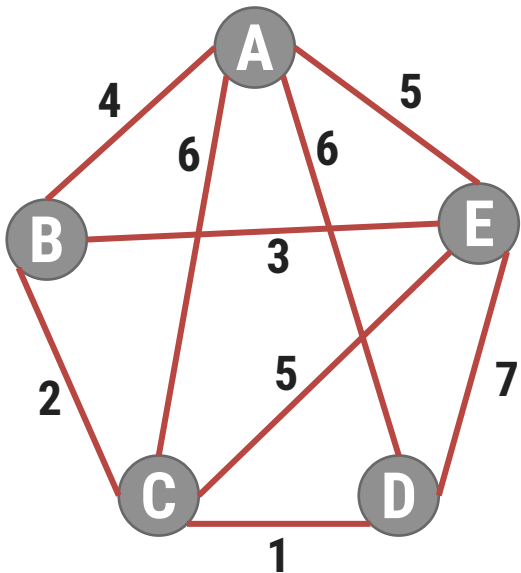


BFS Spanning Tree

Minimum Cost Spanning Tree

- ▶ The **cost of a spanning tree** of a weighted undirected graph is the sum of the costs(weights) of the edges in the spanning tree
- ▶ A **minimum cost spanning tree** is a spanning tree of least cost
- ▶ Two techniques for Constructing minimum cost spanning tree
 - Prim's Algorithm
 - Kruskal's Algorithm

Prims Algorithm

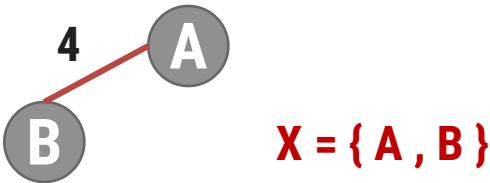


A - B 4	A - D 6	C - E 5
A - E 5	B - E 3	C - D 1
A - C 6	B - C 2	D - E 7

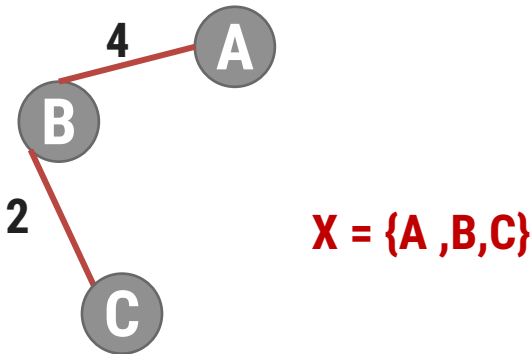
Let X be the set of nodes explored, initially $X = \{ A \}$

A

Step 1: Taking minimum Weight edge of all Adjacent edges of $X=\{A\}$

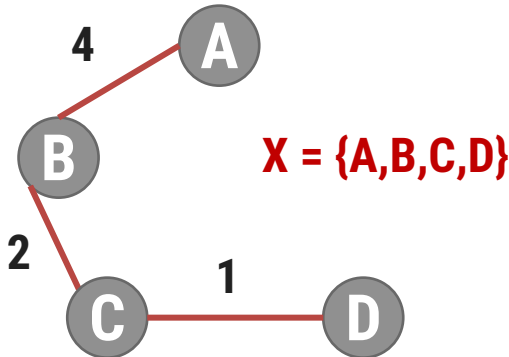


Step 2: Taking minimum weight edge of all Adjacent edges of $X = \{ A , B \}$

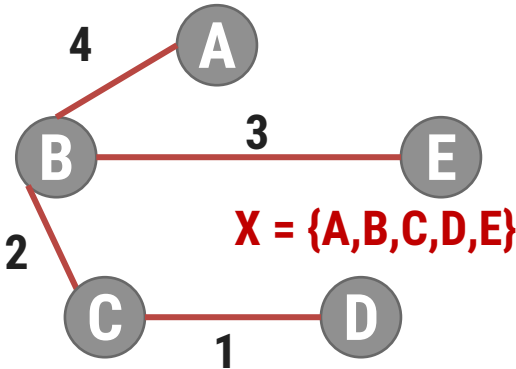


We obtained minimum spanning tree of cost:
 $4 + 2 + 1 + 3 = 10$

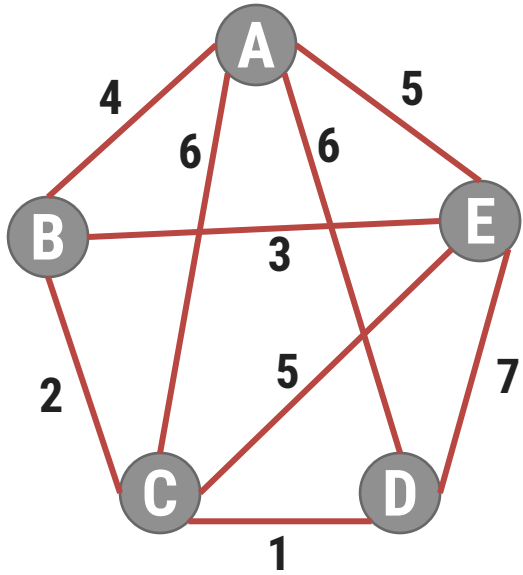
Step 3: Taking minimum weight edge of all Adjacent edges of $X = \{ A , B , C \}$



Step 4: Taking minimum weight edge of all Adjacent edges of $X = \{ A , B , C , D \}$



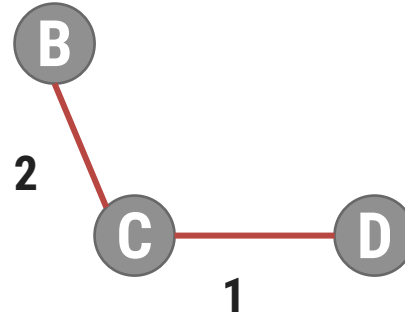
Kruskal's Algorithm



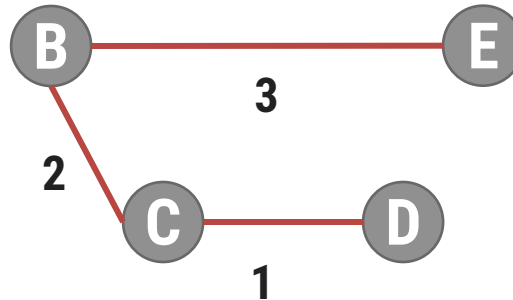
Step 1: Taking min edge (C,D)



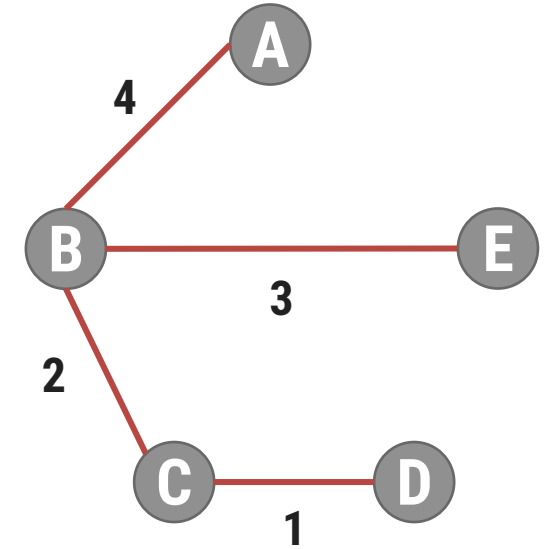
Step 2: Taking next min edge (B,C)



Step 3: Taking next min edge (B,E)

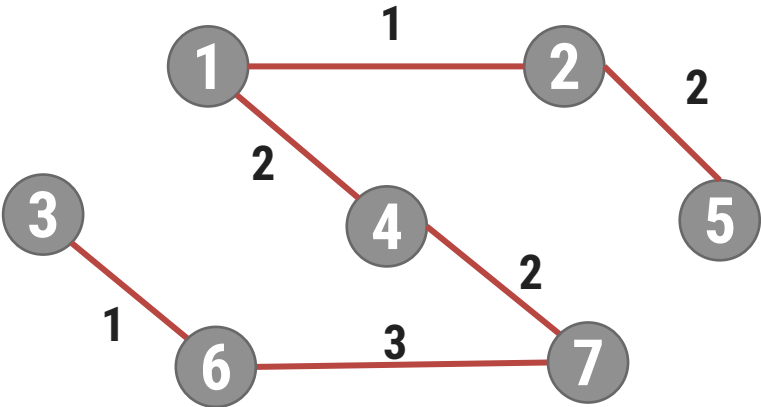
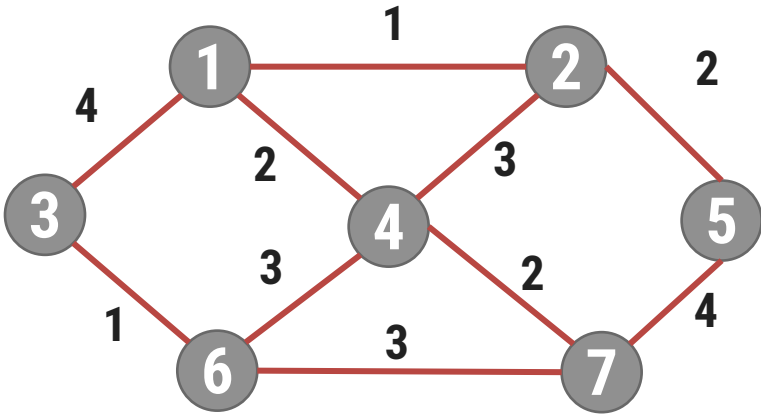
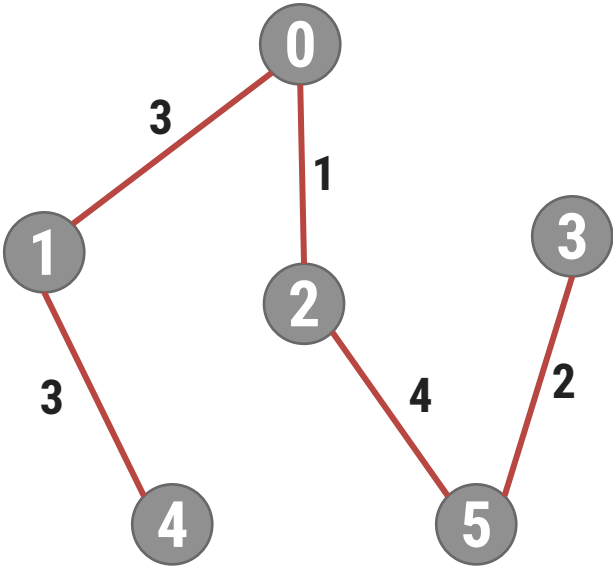
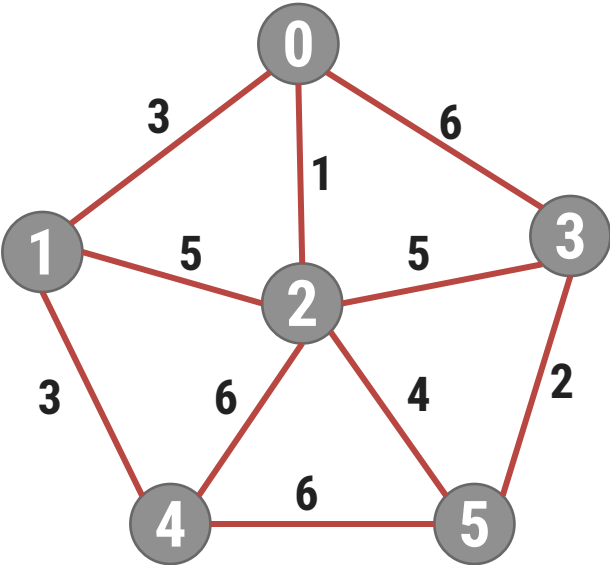


Step 4: Taking next min edge (A,B)

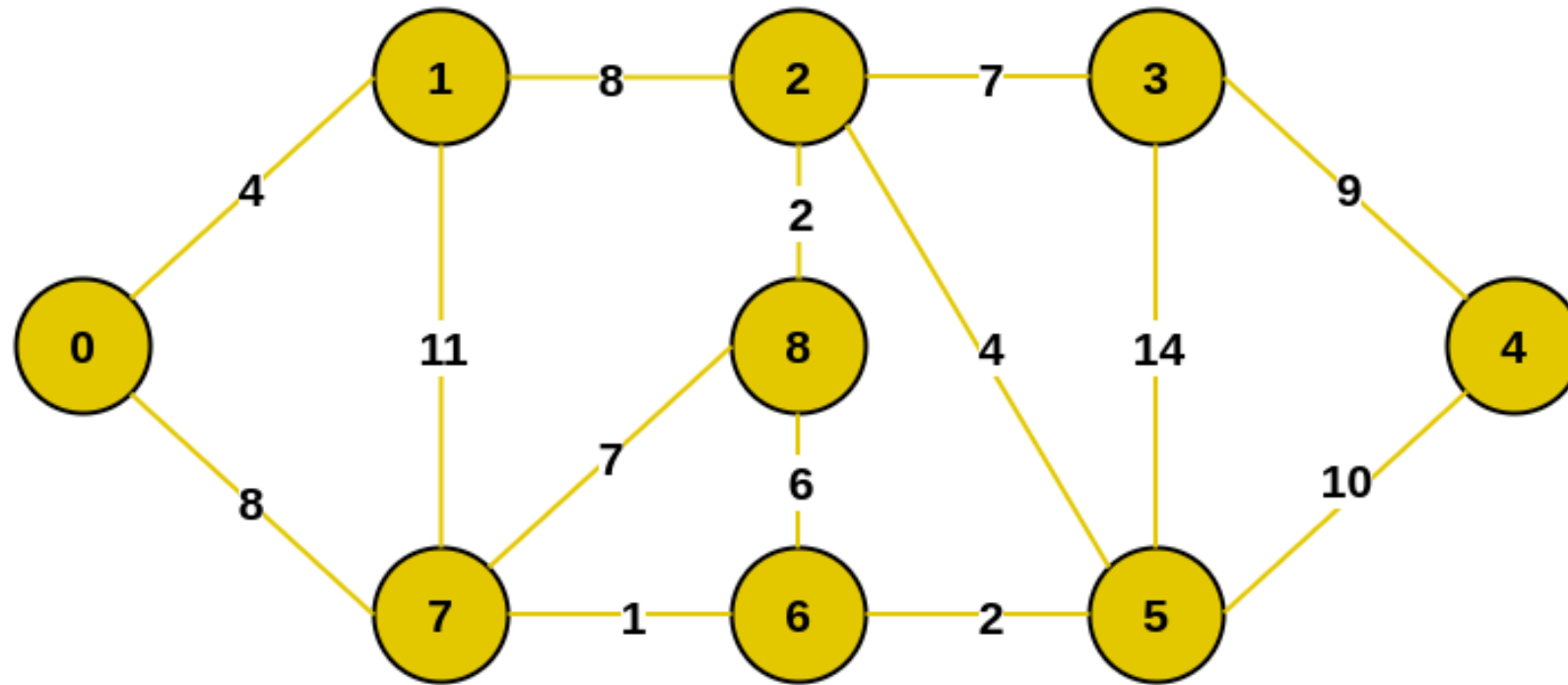


so we obtained minimum
spanning tree of cost:
 $4 + 2 + 1 + 3 = 10$

Construct Minimum Spanning Tree



Example:

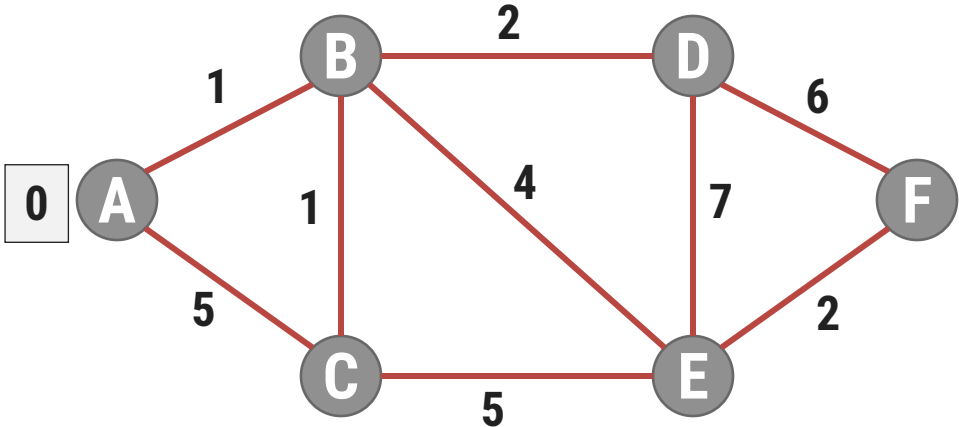


Example of a Graph

Shortest Path Algorithm

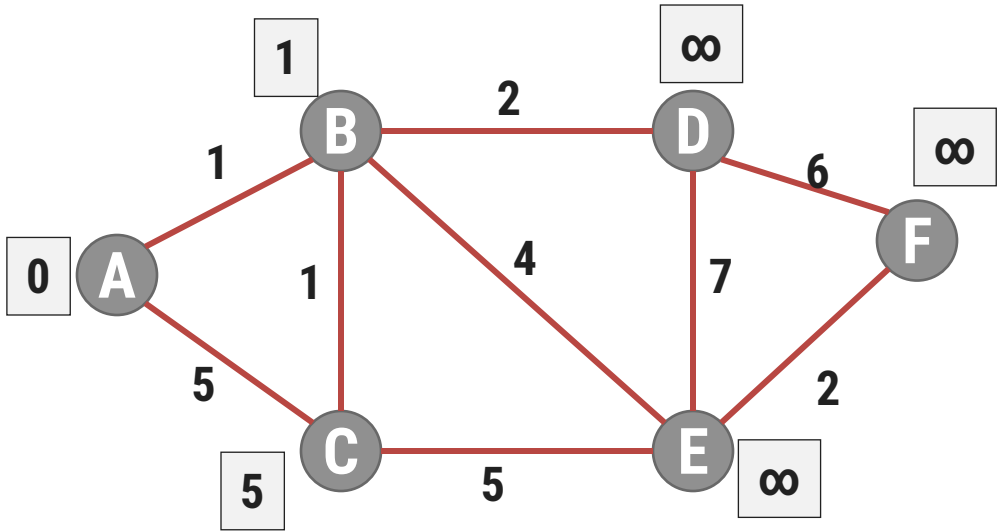
- ▶ Let $G = (V, E)$ be a simple diagraph with **n vertices**
- ▶ The problem is to **find out shortest distance** from a **vertex to all other vertices** of a graph
- ▶ **Dijkstra Algorithm** – it is also called Single Source Shortest Path Algorithm

Dijkstra Algorithm – Shortest Path



	A	B	C	D	E	F
Distance	0	∞	∞	∞	∞	∞
Visited	0	0	0	0	0	0

1st Iteration: Select **Vertex A** with minimum distance



	A	B	C	D	E	F
Distance	0	1	5	∞	∞	∞
Visited	1	0	0	0	0	0

Dijkstra Algorithm – Shortest Path

2nd Iteration: Select **Vertex B** with minimum distance

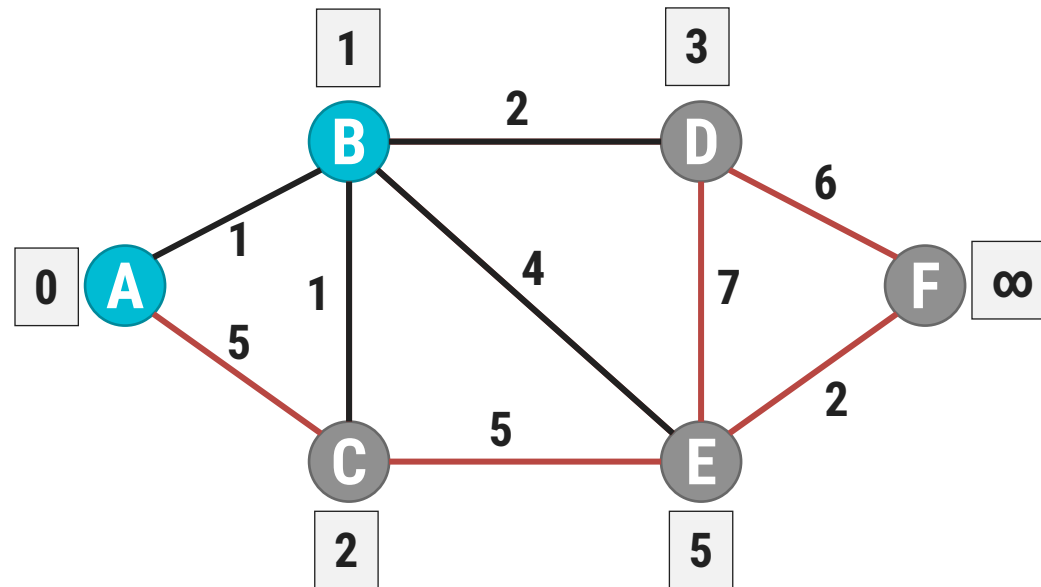
Cost of going to C via B = $\text{dist}[B] + \text{cost}[B][C] = 1 + 1 = 2$

Cost of going to D via B = $\text{dist}[B] + \text{cost}[B][D] = 1 + 2 = 3$

Cost of going to E via B = $\text{dist}[B] + \text{cost}[B][E] = 1 + 4 = 5$

Cost of going to F via B = $\text{dist}[B] + \text{cost}[B][F] = 1 + \infty = \infty$

	A	B	C	D	E	F
Distance	0	1	5	∞	∞	∞
Visited	1	0	0	0	0	0



	A	B	C	D	E	F
Distance	0	1	2	3	5	∞
Visited	1	1	0	0	0	0

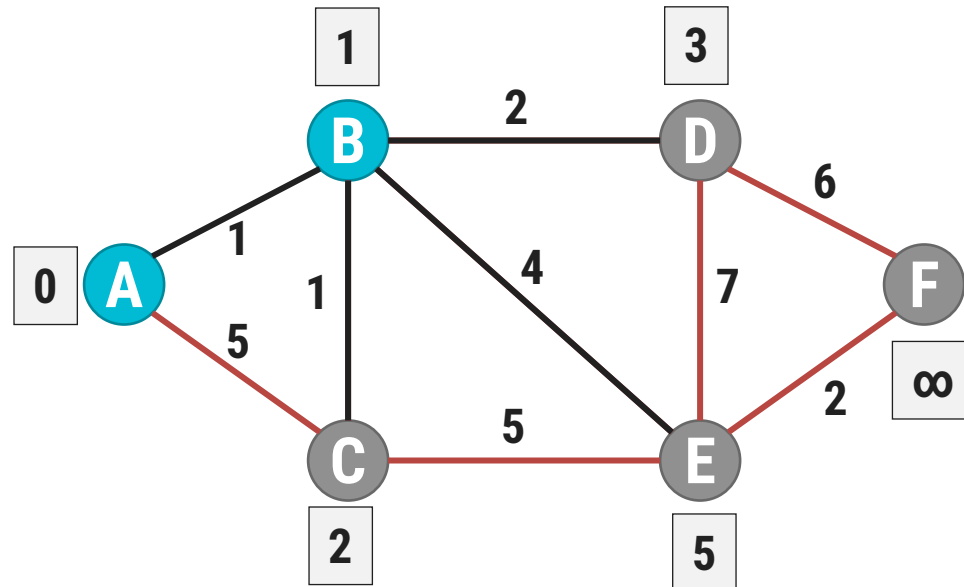
Dijkstra Algorithm – Shortest Path

3rd Iteration: Select **Vertex C** via B with minimum distance

Cost of going to D via C = $\text{dist}[C] + \text{cost}[C][D] = 2 + \infty = \infty$

Cost of going to E via C = $\text{dist}[C] + \text{cost}[C][E] = 2 + 5 = 7$

Cost of going to F via C = $\text{dist}[C] + \text{cost}[C][F] = 2 + \infty = \infty$



	A	B	C	D	E	F
Distance	0	1	2	3	5	∞
Visited	1	1	0	0	0	0

	A	B	C	D	E	F
Distance	0	1	2	3	5	∞
Visited	1	1	1	0	0	0

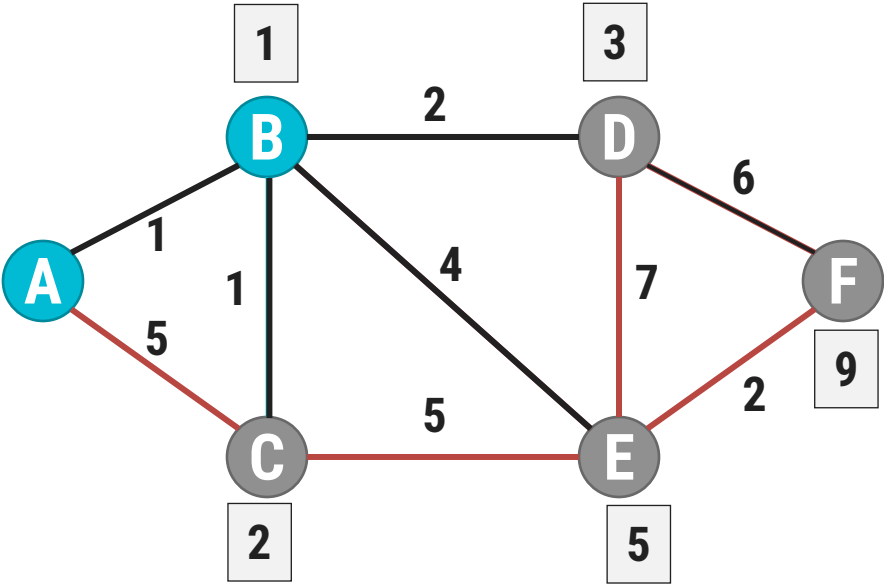
Dijkstra Algorithm – Shortest Path

4th Iteration: Select **Vertex D** via path A - B with minimum distance

Cost of going to E via D = $\text{dist}[D] + \text{cost}[D][E] = 3 + 7 = 10$

Cost of going to F via D = $\text{dist}[D] + \text{cost}[D][F] = 3 + 6 = 9$

	A	B	C	D	E	F
Distance	0	1	2	3	5	∞
Visited	1	1	1	0	0	0



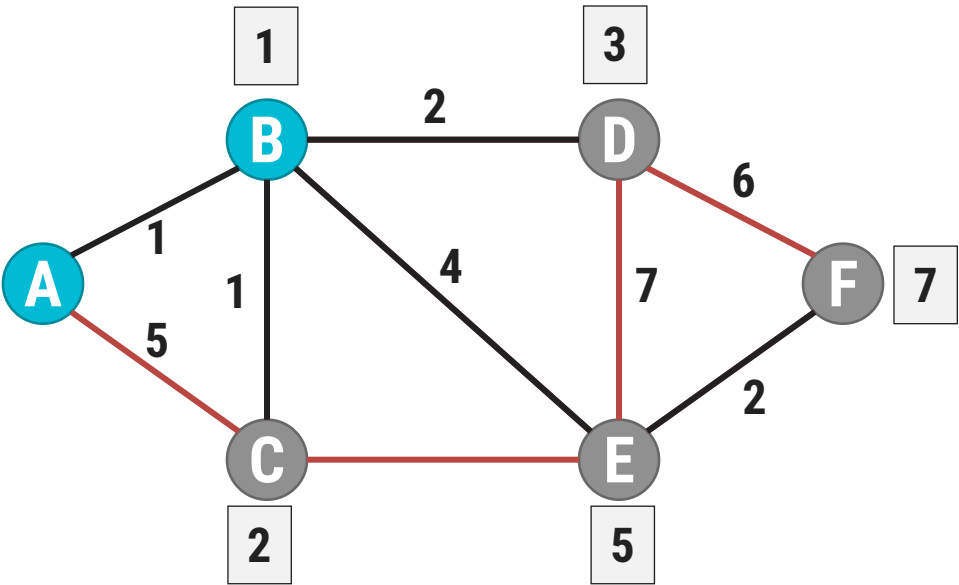
	A	B	C	D	E	F
Distance	0	1	2	3	5	9
Visited	1	1	1	1	0	0

Dijkstra Algorithm – Shortest Path

4th Iteration: Select **Vertex E** via path A – B – E with minimum distance

Cost of going to F via E = $\text{dist}[E] + \text{cost}[E][F] = 5 + 2 = 7$

	A	B	C	D	E	F
Distance	0	1	2	3	5	9
Visited	1	1	1	1	0	0



	A	B	C	D	E	F
Distance	0	1	2	3	5	7
Visited	1	1	1	1	1	0

Shortest Path from A to F is

A → B → E → F = 7

Shortest Path

Find out shortest path from node 0 to all other nodes using Dijkstra Algorithm

